

Níže je připravený studijní materiál k okruhu 27, doplněný a naformátovaný dle požadavků.

---

## 27. Konceptuální návrh relačních databází, základní konstrukty, ER diagram, kardinalita, parcialita, závislost

### Klíčové pojmy

- entita
- atribut
- vztah
- kardinalita
- parcialita
- závislost (slabé entity)
- primární klíč
- cizí klíč

### A. Konceptuální návrh relačních databází

Konceptuální návrh je první fází návrhu databáze, která předchází logickému a fyzickému návrhu. Jeho primárním cílem je zachytit informační požadavky uživatelů a vytvořit srozumitelný model dat, který je nezávislý na konkrétním databázovém systému (DBMS), jenž bude v budoucnu použit (např. MySQL, PostgreSQL, Oracle).

Hlavním nástrojem této fáze je **Entity-Relationship model (ER model)**, který graficky znázorňujeme pomocí **ER diagramu (ERD)**. ER model poskytuje abstraktní pohled na data a jejich vzájemné vztahy v reálném světě.

### B. Základní konstrukty ER modelu

ER model staví na třech základních prvcích (konstruktech):

#### 1. Entita (Entity / Entitní množina)

- Představuje reálný objekt, osobu, místo nebo koncept z reálného světa, který je v systému jednoznačně rozlišitelný a o kterém chceme ukládat data.
- Příklady: `Student` , `Kniha` , `Objednávka` .
- V ER diagramu se typicky znázorňuje jako obdélník.

## 2. Atribut (Attribute)

- Vyjadřuje vlastnost nebo charakteristiku dané entity.
- Příklady: Entita `Student` má atributy `Jméno` , `Příjmení` , `E-mail` , `Datum narození` .
- V ER diagramu se tradičně znázorňuje jako elipsa napojená na příslušný obdélník entity.
- **Klíčový atribut (Identifikátor / Primární klíč):** Atribut (nebo skupina atributů), který jednoznačně identifikuje každou konkrétní instanci entity.
- Příklady: `Rodné číslo` nebo `ID_studenta` .
- V ER diagramu se podtrhává.

## 3. Vztah (Relationship / Vztahová množina)

- Představuje logickou vazbu nebo asociaci mezi dvěma nebo více entitami.
- Příklady: `Student` *studuje* `Předmět` , `Zákazník` *vytváří* `Objednávku` .
- V ER diagramu se tradičně znázorňuje jako kosočtverec ležící na spojnici mezi entitami.

## C. Kardinalita, Parcialita a Závislost

Vztahy mezi entitami nejsou libovolné, ale řídí se přesnými strukturálními omezeními, která popisují pravidla fungování systému.

### 1. Kardinalita (Strukturální poměr vztahu)

- Určuje, kolika instancí jednoho entitního typu se může maximálně účastnit jedna instance druhého entitního typu.
- Rozlišujeme tři základní typy binárních vztahů:

- **1:1 (Jeden k jednomu):** Jedna instance entity A je svázána maximálně s jednou instancí entity B a naopak.
  - Příklad: Každý **Občan** může mít pouze jeden **Cestovní pas** a každý pas patří právě jednomu občanovi.
- **1:N (Jeden k mnoha):** Jedna instance entity A může být svázána s mnoha instancemi entity B, ale instance entity B je svázána maximálně s jednou instancí entity A.
  - Příklad: Jedna **Fakulta** má mnoho **Studentů**, ale každý student studuje pouze na jedné fakultě.
- **M:N (Mnoha k mnoha):** Jedna instance entity A může být svázána s mnoha instancemi entity B a naopak.
  - Příklad: Jeden **Student** může navštěvovat mnoho **Předmětů** a jeden předmět může být navštěvován mnoha studenty.

## 2. Parcialita (Členství / Účast ve vztahu)

- Určuje, zda je účast entity ve vztahu povinná, nebo volitelná.
- **Totální (Povinná) účast:** Každá instance dané entity se musí účastnit alespoň jednoho takového vztahu.
  - Příklad: Každá **Objednávka** musí mít svého **Zákazníka** (nemůže existovat anonymní objednávka).
- **Parciální (Volitelná) účast:** Instance entity se vztahu účastnit může, ale nemusí.
  - Příklad: Každý **Zákazník** registrovaný v e-shopu nemusí mít v danou chvíli vytvořenou žádnou **Objednávku**.

## 3. Závislost (Slabé entitní typy)

- Popisuje situaci, kdy některé entity nemohou existovat samy o sobě, protože jsou existenčně závislé na jiné entitě.
- **Silná entita:** Existuje nezávisle na ostatních objektech a má svůj vlastní jednoznačný klíč (primární klíč).
- **Slabá entita (Identifikačně/Existenčně závislá):** Nemůže existovat bez existence své nadřazené (silné) entity a nemá vlastní klíč, který by ji jednoznačně identifikoval globálně. Její identifikátor je složen z klíče nadřazené entity a vlastního parciálního klíče.

- Příklad: Entita `Místnost` je slabou entitou závislou na entitě `Budova`. `Místnost` s číslem "102" nedává smysl, pokud nevíme, ve které budově se nachází. Smazáním budovy automaticky zanikají i její místnosti.

## Praktické použití

- Návrh informačních systémů
- Komunikace se zákazníkem
- Dokumentace databáze
- Prevence redundance

## Nejčastější chyby u zkoušky

- Příliš brzké řešení SQL bez konceptuálního návrhu.
- Nejasné kardinality.
- Entita zaměněná za atribut.

## Typické zkouškové otázky

### 1. Co je entita, atribut a vztah?

#### ○ Odpověď:

- **Entita** představuje reálný objekt, osobu, místo nebo koncept, o kterém chceme ukládat data (např. `Student`, `Kniha`). V ER diagramu je znázorněna obdélníkem.
- **Atribut** je vlastnost nebo charakteristika entity (např. `Jméno` studenta, `Název` knihy). V ER diagramu je znázorněn elipsou.
- **Vztah** je logická vazba nebo asociace mezi dvěma nebo více entitami (např. `Student` *studuje* `Předmět`). V ER diagramu je znázorněn kosočtvercem.

### 2. Jak se převádí M:N vztah do tabulek?

- **Odpověď:** M:N vztah se v relačním modelu převádí na **spojovací (asociační) tabulku**. Tato nová tabulka obsahuje jako primární klíč složený klíč, který se skládá z primárních klíčů obou entit, mezi nimiž existuje M:N vztah. Tyto klíče se ve spojovací tabulce stávají cizími klíči

odkazujícími na původní entity. Spojovací tabulka může navíc obsahovat vlastní atributy, které popisují samotný vztah (např. u vztahu `Student` `studuje` `Předmět` by spojovací tabulka `Studuje` mohla mít atribut `Známka` ).

### 3. Co je primární a cizí klíč?

- **Odpověď:**

- **Primární klíč** je atribut nebo skupina atributů, která jednoznačně identifikuje každý záznam (řádek) v tabulce. Musí být jedinečný a nesmí obsahovat hodnotu `NULL` . Zajišťuje entitní integritu.
- **Cizí klíč** je atribut nebo skupina atributů v jedné tabulce, která odkazuje na primární klíč v jiné tabulce. Slouží k propojení tabulek a zajišťuje referenční integritu, což znamená, že hodnota cizího klíče musí buď existovat jako primární klíč v odkazované tabulce, nebo být `NULL` (pokud je to povoleno).

## Shrnutí kapitoly

ER model je první krok dobrého návrhu databáze. Pomáhá pochopit doménu před tvorbou tabulek.

Níže je připravený studijní materiál k okruhu "28. Normalizace, normální formy, funkční závislosti, aktualizací anomálie" pro státní závěrečné zkoušky, doplněný a naformátovaný dle zadaných pravidel.

---

## 28. Normalizace, normální formy, funkční závislosti, aktualizací anomálie

### Klíčové pojmy

- **Redundance:** Opakování stejných dat na více místech v databázi.
- **Anomálie:** Nežádoucí vedlejší efekty při operacích vkládání, mazání nebo úpravě dat v špatně navržené databázi.
- **Funkční závislost:** Vztah mezi atributy, kde hodnota jednoho atributu (nebo sady atributů) jednoznačně určuje hodnotu jiného atributu.
- **Determinant:** Atribut nebo sada atributů, které na levé straně funkční závislosti jednoznačně určují atributy na pravé straně.
- **Kandidátní klíč:** Minimální sada atributů, která jednoznačně identifikuje každý řádek v tabulce.
- **Primární klíč:** Vybraný kandidátní klíč, který slouží k jednoznačné identifikaci řádků.
- **1NF (První normální forma):** Základní forma, která vyžaduje atomické atributy a absenci duplicitních řádků.
- **2NF (Druhá normální forma):** Vyžaduje 1NF a plnou funkční závislost neklíčových atributů na celém primárním klíči.
- **3NF (Třetí normální forma):** Vyžaduje 2NF a absenci tranzitivních funkčních závislostí neklíčových atributů na primárním klíči.
- **BCNF (Boyce-Coddova normální forma):** Přísnější forma než 3NF, která vyžaduje, aby každý determinant v tabulce byl kandidátním klíčem.

### A. Aktualizační anomálie

Představ si špatně navrženou tabulku, kde v jednom řádku držíš data o studentovi i o předmětu, který studuje (např. `ID_studenta` , `Jmeno` , `Smerovaci_cislo_TUL` , `Nazev_predmetu` , `Jmeno_vyucujiciho` ). Takový návrh vede ke třem zásadním problémům, kterým říkáme aktualizací anomálie:

- **Anomálie při vkládání (Insertion Anomaly):** Nemůžeme do databáze vložit nový předmět, dokud si ho nezapiše alespoň jeden student (protože bychom neměli primární klíč studenta).
- **Anomálie při mazání (Deletion Anomaly):** Pokud jediný student, který studuje daný předmět, ze školy odejde a my ho smažeme, nechtěně tím z databáze úplně vymažeme i informaci o existenci tohoto předmětu a jeho vyučujícím.
- **Anomálie při úpravě (Update Anomaly):** Pokud se změní název předmětu, musíme tuto informaci vyhledat a přepsat v řádcích všech studentů, kteří ho mají zapsaný. Pokud na nějaký řádek zapomeneme, databáze se dostane do nekonzistentního stavu.

## B. Funkční závislosti

Normalizace je postavena na matematickém konceptu funkčních závislostí.

Říkáme, že atribut  $Y$  je funkčně závislý na atributu  $X$  (zápis:  $X \rightarrow Y$ ), pokud platí, že pro každou konkrétní hodnotu  $X$  existuje v databázi právě jedna odpovídající hodnota  $Y$ .

Příklad: `ID_studenta`  $\rightarrow$  `Jmeno` . Pokud známe ID studenta, jednoznačně tím určíme jeho jméno.

- **Plná funkční závislost:** Atribut  $Y$  je závislý na celém složeném klíči  $X$ , a nikoliv pouze na nějaké jeho části.
- **Tranzitivní funkční závislost:** Pokud  $X \rightarrow Y$  a zároveň  $Y \rightarrow Z$ , pak  $Z$  závisí na  $X$  tranzitivně (zprostředkovaně přes  $Y$ ).

## C. Normální formy (1NF, 2NF, 3NF, BCNF)

Normalizace je proces převodu datové struktury do vyšších normálních forem s cílem odstranit anomálie a duplicity. Každá vyšší forma v sobě implicitně obsahuje podmínky forem nižších.

## 1. První normální forma (1NF)

Tabulka je v 1NF právě tehdy, když:

- Všechny její atributy (sloupce) jsou atomické (nedělitelné). To znamená, že na průsečíku řádku a sloupce nesmí být složený typ ani pole hodnot.
- Tabulka má definovaný primární klíč a neobsahuje duplicitní řádky.

Příklad porušení: Sloupec `Telefon` obsahuje hodnotu "606123456, 777123456". Pro splnění 1NF se tyto hodnoty musí rozdělit do samostatných řádků nebo samostatné tabulky.

## 2. Druhá normální forma (2NF)

Tabulka je ve 2NF právě tehdy, když:

- Splňuje podmínky 1NF.
- Všechny neklíčové atributy jsou plně funkčně závislé na celém primárním klíči. Tento koncept má smysl řešit pouze u tabulek, které mají složený primární klíč (složený z více sloupců).

Příklad porušení: Tabulka reprezentující zápis studentů na předměty se složeným klíčem ( `ID_studenta` , `ID_predmetu` ) obsahuje sloupec `Kancelar_vyucujiciho` . Kancelář však závisí pouze na `ID_predmetu` , nikoliv na studentovi.

Náprava: Atributy, které závisí jen na části klíče, se vyjmou do samostatné tabulky (zde tabulka `Predmet` ).

## 3. Třetí normální forma (3NF)

Tabulka je ve 3NF právě tehdy, když:

- Splňuje podmínky 2NF.
- Žádný neklíčový atribut není tranzitivně závislý na primárním klíči. Jinými slovy, žádný neklíčový sloupec nesmí záviset na jiném neklíčovém sloupci.



Příklad porušení: Tabulka `Student` s primárním klíčem `ID_studenta` obsahuje sloupce `Smerovaci_cislo` a `Mesto`. Platí, že `ID_studenta`  $\rightarrow$  `Smerovaci_cislo`  $\rightarrow$  `Mesto`. Sloupec `Mesto` je závislý na neklíčovém sloupci `Smerovaci_cislo`.

Náprava: Vytvoří se samostatná číselníková tabulka ( `Smerovaci_cislo` , `Mesto` ) a v původní tabulce zůstane pouze cizí klíč `Smerovaci_cislo`.

## 4. Boyce-Coddova normální forma (BCNF)

Tabulka je v BCNF právě tehdy, když:

- Splňuje podmínky 3NF.
- Každá determinantní množina (množina atributů, na které závisí jiné atributy) je kandidátním klíčem.

BCNF je přísnější než 3NF a řeší specifické případy, kdy 3NF nestačí k odstranění redundance a anomálií. K porušení BCNF může dojít, pokud tabulka obsahuje více překrývajících se kandidátních klíčů a existuje funkční závislost, kde determinant není kandidátním klíčem.

Příklad porušení: Představte si tabulku `Student_Predmet_Vyucujici` s atributy ( `Student_ID` , `Predmet` , `Vyucujici` ). Primární klíč je složený z ( `Student_ID` , `Predmet` ). Dále platí, že `Vyucujici` učí pouze jeden `Predmet` (tj. `Vyucujici`  $\rightarrow$  `Predmet` ), ale jeden `Predmet` může učit více `Vyucujici`.

- Tato tabulka je v 3NF, protože neklíčový atribut ( `Vyucujici` ) není tranzitivně závislý na primárním klíči a neexistují částečné závislosti.
- Není však v BCNF, protože `Vyucujici` je determinant ( `Vyucujici`  $\rightarrow$  `Predmet` ), ale není kandidátním klíčem tabulky. To vede k redundanci (stejný předmět se opakuje pro každého studenta, kterého učí daný vyučující) a anomáliím.

Náprava: Rozdělení na tabulky `Student_Predmet_Zapis` ( `Student_ID` , `Predmet` , `Vyucujici` ) a `Vyucujici_Predmet` ( `Vyucujici` , `Predmet` ). V `Student_Predmet_Zapis` by pak `Vyucujici` byl cizí klíč odkazující na `Vyucujici_Predmet`.

## D. Výhody a nevýhody normalizace

### Výhody

- **Odstranění redundance:** Minimalizuje duplicitní data, což šetří místo na disku a snižuje náklady na úložiště.
- **Zajištění konzistence dat:** Eliminace aktualizčních anomálií (vkládání, mazání, úpravy) zajišťuje, že data jsou vždy správná a aktuální.
- **Zlepšení datové kvality:** Díky odstranění anomálií a vynucení funkčních závislostí jsou data přesnější a spolehlivější.
- **Snazší údržba a jasná logická struktura:** Dobře normalizovaná databáze je snáze pochopitelná, spravovatelná a rozšiřitelná.
- **Dlouhodobá udržitelnost IS:** Zjednodušuje budoucí úpravy a rozvoj informačního systému.

## Nevýhody (Kdy se naopak provádí denormalizace)

- **Snížení výkonu při čtení:** Protože jsou data rozsekána do mnoha tabulek, při složitějších dotazech je nutné tabulky neustále propojovat pomocí operace `JOIN`. Spojování velkých tabulek je výpočetně drahé.
- **Zvýšená složitost dotazů:** Dotazy mohou být delší a složitější kvůli nutnosti spojovat více tabulek.

V systémech typu datových skladů (OLAP), kde se data jen analyzují a neupravují, se databáze záměrně navrhují denormalizované (např. schéma hvězdy), aby se maximalizovala rychlost čtení a zjednodušila analytika.

## Nejčastější chyby u zkoušky

- **Mechanické dělení tabulek bez zachování významu dat:** Studenti rozdělují tabulky jen proto, aby dosáhli vyšší normální formy, aniž by chápali funkční závislosti a význam dat.
- **Neznalost kandidátních klíčů:** Nejsou schopni správně identifikovat všechny kandidátní klíče, což je základ pro určení normálních forem.
- **Zaměňování normalizace a indexace:** Normalizace se týká struktury databáze a odstranění redundance, zatímco indexace je o optimalizaci rychlosti vyhledávání.
- **Ignorování BCNF:** Zapomenutí na BCNF, která řeší specifické případy, kde 3NF není dostatečná.

# Typické zkouškové otázky

## 1. Co je funkční závislost?

**Odpověď:** Funkční závislost  $X \rightarrow Y$  znamená, že pro každou konkrétní hodnotu atributu (nebo sady atributů)  $X$  existuje v databázi právě jedna odpovídající hodnota atributu (nebo sady atributů)  $Y$ . Atribut  $X$  se nazývá determinant a jednoznačně určuje atribut  $Y$ .

## 2. Vysvětlete 1NF, 2NF a 3NF.

**Odpověď:**

- **1NF (První normální forma):** Tabulka je v 1NF, pokud všechny její atributy jsou atomické (nedělitelné, tj. neobsahují složené hodnoty nebo opakující se skupiny) a má definovaný primární klíč, který jednoznačně identifikuje každý řádek.
- **2NF (Druhá normální forma):** Tabulka je ve 2NF, pokud je v 1NF a všechny neklíčové atributy jsou plně funkčně závislé na celém primárním klíči. To znamená, že žádný neklíčový atribut nesmí záviset pouze na části složeného primárního klíče.
- **3NF (Třetí normální forma):** Tabulka je ve 3NF, pokud je ve 2NF a žádný neklíčový atribut není tranzitivně závislý na primárním klíči. To znamená, že žádný neklíčový atribut nesmí záviset na jiném neklíčovém atributu.

## 3. Jaké anomálie normalizace odstraňuje?

**Odpověď:** Normalizace odstraňuje tzv. aktualizací anomálie:

- **Anomálie při vkládání (Insertion Anomaly):** Nemožnost vložit nová data o jedné entitě, dokud neexistují data o jiné entitě, na které je závislá.
- **Anomálie při mazání (Deletion Anomaly):** Nechtěné smazání dat o jedné entitě při smazání dat o jiné entitě.
- **Anomálie při úpravě (Update Anomaly):** Nutnost aktualizovat stejná data na více místech, což může vést k nekonzistenci, pokud se některá instance zapomene aktualizovat.

## 4. Co je BCNF a jaký je její hlavní rozdíl oproti 3NF?

**Odpověď:** BCNF (Boyce-Coddova normální forma) je přísnější normální forma než 3NF. Tabulka je v BCNF, pokud je v 3NF a každá determinantní množina (tj. každý

atribut nebo sada atributů, které funkčně určují jiné atributy) je kandidátním klíčem. Hlavní rozdíl oproti 3NF spočívá v tom, že 3NF povoluje tranzitivní závislosti, pokud determinant není superklíč, zatímco BCNF vyžaduje, aby každý determinant byl kandidátním klíčem, čímž řeší i některé specifické případy redundance a anomálií, které 3NF nemusí pokrýt, zejména u tabulek s více překrývajícími se kandidátními klíči.

## Shrnutí kapitoly

Normalizace je klíčový proces v návrhu relačních databází, který pomocí funkčních závislostí převádí datovou strukturu do vyšších normálních forem (1NF, 2NF, 3NF, BCNF). Jejím hlavním cílem je odstranit redundanci dat a eliminovat aktualizací anomálie (při vkládání, mazání a úpravě), čímž se zajišťuje datová konzistence, integrita a usnadňuje údržba systému. Ačkoliv normalizace může vést ke snížení výkonu při čtení dat kvůli nutnosti spojování více tabulek, její výhody v oblasti datové kvality a udržitelnosti obvykle převažují, zejména v transakčních systémech. V analytických systémech se naopak často využívá denormalizace pro optimalizaci rychlosti dotazů.

---

Zde je detailní a kompletní studijní materiál k okruhu "29. Relační model, základní konstrukty, realizace vztahů v relačním modelu, integritní omezení", připravený pro státní závěrečné zkoušky.

---

## 29. Relační model, základní konstrukty, realizace vztahů v relačním modelu, integritní omezení

### Klíčové pojmy

- Relace (tabulka)
  - Atribut (sloupec)
  - N-tice (řádek)
  - Doména
  - Schéma relace
  - Instance relace
  - Stupeň relace
  - Kardinalita relace
  - Primární klíč (PK)
  - Kandidátní klíč
  - Složený klíč
  - Cizí klíč (FK)
  - Vazební (asociativní) tabulka
  - Entitní integrita
  - Referenční integrita
  - Doménová integrita
  - Uživatelská (Business) integrita
  - Referenční akce ( CASCADE , SET NULL , RESTRICT / NO ACTION , SET DEFAULT )
- 

### A. Základní konstrukty relačního modelu

Relační datový model, navržený E. F. Coddem v roce 1970, je založen na matematickém pojmu relace. V praxi je relace reprezentována jako tabulka. Tento model je základem většiny moderních databázových systémů díky své jednoduchosti a matematické robustnosti, která umožňuje formální definici datové integrity a manipulace s daty.

Mezi základní konstrukty a terminologii relačního modelu patří:

- **Relace (Tabulka)**

- Dvourozměrná datová struktura, která se skládá z pojmenovaných sloupců (atributů) a neuspořádané množiny řádků (n-tic).
- Každá relace má unikátní název.
- Relace představuje množinu n-tic, což znamená, že neobsahuje duplicitní řádky.

- **Atribut (Sloupec)**

- Pojmenovaná vlastnost, která uchovává data v tabulce.
- Každý atribut má definovaný svůj **datový doménový rozsah (doménu)**, což je množina povolených hodnot (např. `INT` , `VARCHAR(255)` , `DATE` , `BOOLEAN` ).
- Hodnoty atributů by měly být **atomické**, tj. nedělitelné (např. jméno by nemělo obsahovat křestní jméno i příjmení v jednom sloupci, pokud je potřeba s nimi pracovat odděleně).

- **N-tice (Řádek, Záznam)**

- Konkrétní instance dat v tabulce, představující jeden záznam.
- Každá n-tice je unikátní v rámci relace (díky primárnímu klíči).
- Pořadí řádků v relačním modelu je teoreticky nedůležité; databázový systém je může ukládat a vracet v libovolném pořadí, pokud není explicitně požadováno řazení.

- **Doména**

- Množina všech povolených hodnot pro daný atribut. Definuje datový typ (např. `INTEGER` , `STRING` , `DATE` ) a případně další omezení (např. rozsah hodnot, formát).

- **Schéma relace**

- Logická definice struktury relace, která zahrnuje název relace a seznam jejích atributů s odpovídajícími doménami.
- Příklad: `Student (ID: INT, Jméno: VARCHAR(100), DatumNarození: DATE)` .

- **Instance relace**

- Aktuální obsah relace (tabulky) v daném čase, tj. konkrétní množina n-tic, které splňují schéma relace.

- **Stupeň relace (Arity)**

- Počet atributů (sloupců) v tabulce.

- **Kardinalita relace**

- Počet n-tic (řádků) v tabulce.

---

## B. Realizace vztahů v relačním modelu

V relačním modelu neexistuje přímý konstrukční prvek pro "vztah" jako v ER diagramech. Vztahy v ER diagramu jsou logické vazby mezi entitami, zatímco v relačním modelu se tyto logické vztahy realizují fyzicky pomocí provázání hodnot klíčů v tabulkách.

### 1. Primární klíč (PK - Primary Key)

- **Definition:** Atribut nebo minimální množina atributů, která jednoznačně identifikuje každý řádek v tabulce.
- **Vlastnosti:**
  - **Unikátnost:** Hodnota primárního klíče musí být v rámci tabulky unikátní pro každý řádek.
  - **NOT NULL:** Hodnota primárního klíče nesmí být nikdy prázdná ( `NULL` ).
  - **Stabilita:** Ideálně by se hodnota primárního klíče neměla měnit v průběhu času.
- **Kandidátní klíč:** Jakákoli minimální množina atributů, která může jednoznačně identifikovat řádek. Z množiny kandidátních klíčů se jeden

vybere jako primární klíč.

- **Složený primární klíč:** Primární klíč tvořený dvěma nebo více atributy.  
Příklad: V tabulce `Zapis_Predmetu` může být složený PK  
(`id_studenta_fk`, `id_predmetu_fk`) .

## 2. Cizí klíč (FK - Foreign Key)

- **Definice:** Atribut (nebo skupina atributů) v jedné tabulce (tzv. odkazující tabulka), který odkazuje na primární klíč (nebo kandidátní klíč) v tabulce jiné (tzv. odkazovaná/rodičovská tabulka).
- **Význam:** Cizí klíč je nástroj, kterým fyzicky propojujeme tabulky a vytváříme logické vztahy mezi nimi. Zajišťuje referenční integritu.

## 3. Přepis vztahů z ER diagramu do tabulek

### ○ Vztah 1:N (Jeden k mnoha)

- **Realizace:** Primární klíč z tabulky na straně "1" (rodičovská tabulka) se vezme a vloží se jako cizí klíč (FK) do tabulky na straně "N" (potomkovská tabulka).
- **Příklad:** Vztah mezi `Fakulta` (1) a `Student` (N).
- Tabulka `Fakulta` má primární klíč `id_fakulty` .
- Do tabulky `Student` se přidá sloupec `id_fakulty_fk` jako cizí klíč, který odkazuje na `id_fakulty` v tabulce `Fakulta` .
- Schéma:
  - `Fakulta (id_fakulty PK, nazev)`
  - `Student (id_studenta PK, jmeno, id_fakulty_fk FK)`

### ○ Vztah M:N (Mnoha k mnoha)

- **Realizace:** V relačním modelu nelze zapsat přímo mezi dvě tabulky. Musí se rozbít vytvořením třetí, tzv. **vazební (asociativní) tabulky**.
- Vazební tabulka obsahuje minimálně dva cizí klíče, které odkazují na primární klíče obou původních tabulek. Tyto dva cizí klíče spolu obvykle tvoří **složený primární klíč** vazební tabulky, čímž se zajišťuje unikátnost každé kombinace. Vazební tabulka může



obsahovat i další atributy, které popisují samotný vztah (např. datum zápisu, známka).

- **Příklad:** Vztah mezi `Student` (M) a `Predmet` (N).
- Vytvoří se vazební tabulka `Zapis_Predmetu` .
- Ta obsahuje `id_studenta_fk` (cizí klíč na `Student` ) a `id_predmetu_fk` (cizí klíč na `Predmet` ).
- Složený primární klíč tabulky `Zapis_Predmetu` je `(id_studenta_fk, id_predmetu_fk)` .
- Schéma:
  - `Student` (`id_studenta` PK, `jmeno`)
  - `Predmet` (`id_predmetu` PK, `nazev`)
  - `Zapis_Predmetu` (`id_studenta_fk` FK, `id_predmetu_fk` FK, `datum_zapisu`, `znamka`)

#### ○ Vztah 1:1 (Jeden k jednomu)

- **Realizace:** Primární klíč jedné tabulky se vloží jako cizí klíč do druhé tabulky. Na tento cizí klíč se navíc uvalí podmínka **unikátnosti** ( `UNIQUE` ), aby na jeden řádek první tabulky mohl odkazovat maximálně jeden řádek druhé tabulky.
- **Kdy použít:** Často se používá pro oddělení volitelných atributů, pro bezpečnostní důvody (např. citlivá data v samostatné tabulce s omezeným přístupem) nebo pro optimalizaci výkonu (rozdělení široké tabulky na menší, častěji používané části).
- **Příklad:** Tabulka `Osoba` a `DetailOsoby` .
- Schéma:
  - `Osoba` (`id_osoby` PK, `jmeno`, `prijmeni`)
  - `DetailOsoby` (`id_osoby_fk` PK, `UNIQUE`, `adresa`, `telefon`)
  - Zde `id_osoby_fk` je zároveň primárním klíčem tabulky `DetailOsoby` a cizím klíčem odkazujícím na `Osoba` .

---

## C. Integritní omezení

Integritní omezení jsou pravidla a podmínky, které musí databáze za všech okolností povinně vynucovat, aby byla zajištěna přesnost, konzistence a správnost dat. Pokud se aplikace pokusí tato pravidla porušit (např. vložit nevalidní data),

databázový systém operaci zamítne a vrátí chybu. Jsou klíčové pro udržení kvality dat a spolehlivosti informačního systému.

U státnic je bezpodmínečně nutné vyjmenovat tyto 4 základní úrovně integrity:

## 1. Entitní integrita

- **Pravidlo:** Každá tabulka musí mít definovaný primární klíč. Hodnota primárního klíče musí být v rámci tabulky unikátní a nesmí obsahovat hodnotu `NULL` (prázdnost).
- **Význam:** Zajišťuje, že v tabulce dokážeme každý záznam jednoznačně identifikovat a adresovat. Bez entitní integrity by nebylo možné spolehlivě odkazovat na konkrétní řádky, což by znemožnilo fungování cizích klíčů a relačního modelu jako celku.

## 2. Referenční integrita

- **Pravidlo:** Hodnota cizího klíče v odkazující tabulce (potomkovské) musí buď přesně odpovídat existující hodnotě primárního klíče v odkazované (rodičovské) tabulce, nebo musí být `NULL` (pokud je cizí klíč povolen jako `NULL`).
- **Význam:** Zabraňuje vzniku "osiřelých" záznamů, tj. záznamů, které odkazují na neexistující rodičovské záznamy. Příklad: Nelze zapsat studenta na fakultu s ID, které v tabulce fakult vůbec neexistuje.
- **Referenční akce (Co se stane, když se smaže nebo aktualizuje rodičovský záznam):** Při definici cizího klíče určujeme, jak se má systém zachovat, pokud smažeme ( `ON DELETE` ) nebo aktualizujeme ( `ON UPDATE` ) řádek v rodičovské tabulce, na který jiné tabulky odkazují:
  - `RESTRICT / NO ACTION` (Výchozí chování):
    - `RESTRICT` : Smazání nebo aktualizace rodičovského záznamu je okamžitě zakázáno, dokud na něj existují odkazy v potomkovské tabulce.
    - `NO ACTION` : Smazání nebo aktualizace rodičovského záznamu je zakázáno, pokud na něj existují odkazy v potomkovské tabulce, ale kontrola se provádí až na konci transakce. V praxi se často chovají podobně.
  - `CASCADE` :

- Smazání nebo aktualizace se šíří kaskádově. Smazáním fakulty se automaticky smažou i všichni studenti na ní zapsaní. Aktualizací ID fakulty se automaticky aktualizuje `id_fakulty_fk` u všech studentů.
- `SET NULL` :
  - Rodičovský záznam se smaže nebo aktualizuje a u odkazujících potomků se v cizím klíči hodnota přepíše na `NULL` . To je možné pouze v případě, že cizí klíč v potomkovské tabulce není definován jako `NOT NULL` .
- `SET DEFAULT` :
  - Rodičovský záznam se smaže nebo aktualizuje a u odkazujících potomků se v cizím klíči hodnota přepíše na předem definovanou výchozí hodnotu.

### 3. Doménová integrita

- **Pravidlo:** Hodnoty v konkrétním sloupci musí přesně odpovídat definovanému datovému typu, povolenému rozsahu nebo seznamu hodnot.
- **Nástroje:**
  - **Výběr datového typu:** Např. `INT` pro celá čísla, `VARCHAR(255)` pro text, `DATE` pro datum.
  - **Příznaky** `NOT NULL` : Sloupec nesmí být prázdný.
  - **Omezující podmínky** `CHECK` : Např. `CHECK (vek >= 18)` zajistí, že věk bude vždy alespoň 18.
  - `DEFAULT` **hodnota:** Pokud není hodnota explicitně zadána při vkládání, použije se výchozí hodnota.

### 4. Uživatelská (Business) integrita

- **Pravidlo:** Specifická pravidla definovaná vývojářem na základě reality daného podniku či systému, která nelze vyjádřit pouhým datovým typem nebo základními integritními omezeními.
- **Realizace:**
  - `UNIQUE` **constraint:** Zajišťuje, že hodnoty v daném sloupci (nebo skupině sloupců) jsou unikátní, i když se nejedná o primární klíč (např. emailová adresa uživatele).

- **Spouště (Triggery):** Automaticky spouštěný kód v databázi při určitých událostech ( `INSERT` , `UPDATE` , `DELETE` ), který může vynucovat složitější pravidla.
  - **Uložené procedury (Stored Procedures):** Předkompilované sady SQL příkazů, které mohou obsahovat složitou obchodní logiku a zajistit, že data jsou vkládána a modifikována konzistentním způsobem.
    - **Příklad:** "Student si může zapsat maximálně 5 předmětů za semestr." (To by se řešilo triggerem nebo v aplikační logice, která by volala uloženou proceduru).
- 

## Praktické použití

- **Ukládání strukturovaných dat:** Relační databáze jsou základem pro většinu podnikových informačních systémů, e-commerce platform, bankovníctví, zdravotnictví a mnoha dalších aplikací, kde je potřeba spolehlivě ukládat a spravovat strukturovaná data.
  - **Zajištění konzistence a kvality dat:** Integritní omezení jsou klíčová pro udržení spolehlivosti a přesnosti dat v dlouhodobém horizontu, minimalizují chyby a zajišťují, že data odpovídají reálnému světu.
  - **Podpora komplexních dotazů:** Relační model umožňuje efektivní dotazování a manipulaci s daty pomocí deklarativního jazyka SQL, což usnadňuje získávání informací a analýzu dat.
  - **Modularizace a organizace dat:** Rozdělení dat do logických tabulek a definování vztahů usnadňuje správu, pochopení datové struktury a vývoj aplikací.
  - **Základ pro Business Intelligence (BI) a reporting:** Konzistentní a dobře strukturovaná data jsou nezbytná pro analýzu, tvorbu reportů a rozhodování v podniku.
- 

## Nejčastější chyby u zkoušky

- **Neznalost rozdílu mezi logickým ER modelem a fyzickým relačním modelem:** Zaměňování pojmů "entita" a "tabulka", "vztah" a "cizí klíč".

- **Chybné převody M:N vztahů:** Zapomenutí na vazební tabulku nebo nesprávná definice jejího primárního klíče (např. definování pouze jednoho sloupce jako PK).
- **Neznalost vlastností primárního klíče:** Zejména `NOT NULL` a unikátnost.
- **Nepochopení referenčních akcí:** Neznalost rozdílů mezi `CASCADE`, `SET NULL` a `RESTRICT / NO ACTION` a kdy kterou použít.
- **Ignorování integritních omezení:** Představa, že integritu zajistí pouze aplikační logika, nikoli databáze, což vede k nespolehlivým systémům.
- **Záměna doménové a uživatelské integrity:** Nepochopení, co spadá pod základní typy integrity a co pod specifická obchodní pravidla.

---

## Typické zkouškové otázky

1. **Popište základní konstrukty relačního modelu a vysvětlete jejich význam.**

**Odpověď:** Základní konstrukty relačního modelu jsou:

- **Relace (Tabulka):** Dvourozměrná datová struktura pro ukládání dat, skládající se z pojmenovaných sloupců (atributů) a neuspořádané množiny řádků (n-tic). Je to základní jednotka pro organizaci dat.
- **Atribut (Sloupec):** Pojmenovaná vlastnost, která uchovává data v tabulce. Každý atribut má definovanou doménu (množinu povolených hodnot). Atributy definují strukturu dat.
- **N-tice (Řádek, Záznam):** Konkrétní instance dat v tabulce, představující jeden záznam. Každá n-tice je unikátní a obsahuje hodnoty pro všechny atributy relace.
- **Doména:** Množina všech povolených hodnot pro daný atribut, definující jeho datový typ a případně rozsah. Zajišťuje datovou integritu na úrovni jednotlivých hodnot.
- **Schéma relace:** Název relace a seznam jejích atributů s doménami, popisující strukturu tabulky.
- **Instance relace:** Aktuální obsah relace (množina n-tic) v daném čase.

2. **Jak se realizují vztahy 1:N a M:N v relačním modelu? Uveďte příklady.**

**Odpověď:** V relačním modelu se vztahy realizují pomocí provázání klíčů:

• **Vztah 1:N (Jeden k mnoha):** Realizuje se tak, že se **primární klíč** z tabulky na straně "1" (rodičovská tabulka) vloží jako **cizí klíč** do tabulky na straně "N" (potomkovská tabulka).

◦ **Příklad:** Vztah mezi `Fakulta` (1) a `Student` (N). Tabulka `Fakulta` má primární klíč `id_fakulty`. Do tabulky `Student` se přidá sloupec `id_fakulty_fk`, který je cizím klíčem odkazujícím na `id_fakulty` v tabulce `Fakulta`.

◦ Schéma:

◦ `Fakulta` (`id_fakulty` PK, `nazev`)

◦ `Student` (`id_studenta` PK, `jmeno`, `id_fakulty_fk` FK)

• **Vztah M:N (Mnoha k mnoha):** Realizuje se vytvořením **třetí, tzv. vazební (asociativní) tabulky**. Tato vazební tabulka obsahuje minimálně dva cizí klíče, které odkazují na primární klíče obou původních tabulek. Tyto dva cizí klíče spolu obvykle tvoří **složený primární klíč** vazební tabulky.

◦ **Příklad:** Vztah mezi `Student` (M) a `Predmet` (N). Vytvoří se vazební tabulka `Zapis_Predmetu`. Ta obsahuje `id_studenta_fk` (cizí klíč na `Student`) a `id_predmetu_fk` (cizí klíč na `Predmet`). Složený primární klíč tabulky `Zapis_Predmetu` je (`id_studenta_fk`, `id_predmetu_fk`).

◦ Schéma:

◦ `Student` (`id_studenta` PK, `jmeno`)

◦ `Predmet` (`id_predmetu` PK, `nazev`)

◦ `Zapis_Predmetu` (`id_studenta_fk` FK, `id_predmetu_fk` FK, `datum_zapisu`)

### 3. Vyjmenujte a vysvětlete základní typy integritních omezení v relačním modelu.

**Odpověď:** Základní typy integritních omezení jsou:

• **Entitní integrita:** Pravidlo, že každá tabulka musí mít primární klíč, jehož hodnoty jsou unikátní a nejsou `NULL`. Zajišťuje jednoznačnou identifikaci každého záznamu v tabulce.

• **Referenční integrita:** Pravidlo, že hodnota cizího klíče v odkazující tabulce musí buď odpovídat existující hodnotě primárního klíče v odkazované tabulce, nebo musí být `NULL` (pokud je povoleno). Zabraňuje vzniku "osiřelých" záznamů.

• **Doménová integrita:** Pravidlo, že hodnoty v konkrétním sloupci musí odpovídat definovanému datovému typu, povolenému rozsahu nebo

seznamu hodnot. Vynucuje správnost dat na úrovni jednotlivých atributů (např. pomocí datových typů, `NOT NULL`, `CHECK` podmínek).

- **Uživatelská (Business) integrita:** Specifická pravidla definovaná na základě obchodní logiky nebo reality systému, která nelze vynutit předchozími typy integrity. Realizuje se pomocí `UNIQUE` omezení, spouští (triggerů) nebo uložených procedur. Příkladem je omezení počtu zapsaných předmětů pro studenta.

#### 4. Vysvětlete referenční akce `CASCADE`, `SET NULL` a `RESTRICT` při definici cizího klíče.

**Odpověď:** Referenční akce určují chování databázového systému při pokusu o smazání ( `ON DELETE` ) nebo aktualizaci ( `ON UPDATE` ) záznamu v rodičovské tabulce, na který odkazují cizí klíče v potomkovské tabulce:

- `CASCADE` : Pokud je rodičovský záznam smazán nebo aktualizován, databáze automaticky smaže nebo aktualizuje všechny související záznamy v potomkovské tabulce. Např. smazání fakulty smaže všechny studenty této fakulty.
- `SET NULL` : Pokud je rodičovský záznam smazán nebo aktualizován, databáze nastaví hodnotu cizího klíče v souvisejících záznamech potomkovské tabulky na `NULL` . To je možné pouze, pokud cizí klíč v potomkovské tabulce není definován jako `NOT NULL` . Např. smazání fakulty nastaví `id_fakulty_fk` u studentů na `NULL` .
- `RESTRICT / NO ACTION` : Databáze zabrání smazání nebo aktualizaci rodičovského záznamu, pokud na něj existují související záznamy v potomkovské tabulce. `RESTRICT` provádí kontrolu okamžitě, `NO ACTION` na konci transakce. Např. nelze smazat fakultu, dokud na ní studuje alespoň jeden student.

---

## Shrnutí kapitoly

Relační model je matematicky jednoduchý a prakticky silný způsob uložení strukturovaných dat. Jeho základem jsou relace (tabulky) s atributy (sloupci) a n-ticemi (řádky). Vztahy mezi tabulkami se realizují pomocí primárních a cizích klíčů, přičemž pro M:N vztahy je nutná vazební tabulka. Integritní omezení – entitní, referenční, doménová a uživatelská – zajišťují konzistenci, správnost a kvalitu dat v

databázi, což je klíčové pro spolehlivost informačních systémů. Pochopení těchto principů je nezbytné pro správný návrh a správu databází.



Níže naleznete studijní materiál k okruhu "30. CRUD operace...", doplněný a naformátovaný.

Tento studijní materiál je připraven pro studenty aplikované informatiky k okruhu státních závěrečných zkoušek č. 30. Materiál kombinuje studentův výtah s oficiálním sylabem a požadavky fakulty, aby poskytl komplexní a přesné informace.

## 30. CRUD operace, SQL DDL, SQL DML, SQL dotazy selekce, projekce, agregační funkce, množinové operace, typy spojení, vnořené dotazy. Indexy, pohledy, uložené procedury a funkce, trigger. Transakce, ACID vlastnosti, úrovně izolace transakcí.

### A. Co jsou operace CRUD?

Zkratka **CRUD** reprezentuje čtyři základní operace, které musí každá perzistentní datová struktura (databáze, ale i REST API nebo souborový systém) umět provést s datovými záznamy. Tyto operace tvoří základ interakce s daty.

V následující tabulce je vidět, jak tyto konceptuální operace mapují přímo na konkrétní příkazy jazyka SQL:

| CRUD Operace | Popis                                     | SQL Příkaz |
|--------------|-------------------------------------------|------------|
| Create       | Vytvoření nového datového záznamu.        | INSERT     |
| Read         | Čtení (vyhledávání) existujících záznamů. | SELECT     |
| Update       | Modifikace existujících záznamů.          | UPDATE     |
| Delete       | Smazání existujících záznamů.             | DELETE     |

## B. Rozdělení jazyka SQL: DDL a DML

Jazyk SQL (Structured Query Language) se dělí do několika logických podskupin podle toho, zda manipulujeme se samotnou strukturou databáze, nebo s daty uvnitř ní.

### 1. SQL DDL (Data Definition Language – Jazyk pro definici dat)

Slouží k vytváření, úpravě a mazání samotné struktury (schématu) databázových objektů, jako jsou tabulky, pohledy, indexy, uložené procedury, funkce, triggerly nebo databáze.

- **CREATE** : Vytvoří nový objekt v databázi.
  - Příklad: `CREATE TABLE student (id INT PRIMARY KEY, jmeno VARCHAR(100), vek INT);`
  - Příklad: `CREATE DATABASE moje_databaze;`
- **ALTER** : Změní strukturu existujícího objektu.
  - Příklad: `ALTER TABLE student ADD COLUMN email VARCHAR(255);`
  - Příklad: `ALTER TABLE student MODIFY COLUMN vek SMALLINT;`
- **DROP** : Kompletně smaže objekt z databáze i s jeho strukturou a daty.
  - Příklad: `DROP TABLE student;`
  - Příklad: `DROP DATABASE moje_databaze;`
- **TRUNCATE** : Rychle odstraní všechny řádky z tabulky, ale zachová její strukturu. Je to DDL příkaz, protože operace je logována jako DDL a nelze ji vrátit zpět pomocí `ROLLBACK` (v některých systémech).
  - Příklad: `TRUNCATE TABLE student;`

### 2. SQL DML (Data Manipulation Language – Jazyk pro manipulaci s daty)

Slouží k práci s konkrétními záznamy (daty) uvnitř již vytvořených struktur.

- **SELECT** : Dotazování a čtení dat z jedné nebo více tabulek.
  - Příklad: `SELECT jmeno, email FROM student WHERE vek > 20;`
- **INSERT** : Vkládání nových řádků do tabulky.
  - Příklad: `INSERT INTO student (id, jmeno, vek) VALUES (1, 'Jan Novák', 22);`

- **UPDATE** : Modifikace hodnot v existujících řádcích tabulky na základě podmínky.
  - Příklad: `UPDATE student SET vek = 23 WHERE jmeno = 'Jan Novák';`
- **DELETE** : Mazání konkrétních řádků z tabulky na základě podmínky.
  - Příklad: `DELETE FROM student WHERE vek < 20;`

## C. Základní operace relační algebry v SQL

Při psaní dotazu `SELECT` provádíme operace vycházející z relační algebry:

- **Selekce (Restrikce / Horizontální filtrace)**: Výběr pouze těch řádků (n-tic), které splňují zadanou podmínku. V SQL se realizuje pomocí klauzule `WHERE` .
  - Příklad: Vyber studenty, kteří jsou z Prahy. `sql SELECT`  
`FROM student WHERE mesto = 'Praha';`
- **Projekce (Vertikální filtrace)**: Výběr pouze konkrétních sloupců (atributů), které chceme ve výsledku zobrazit. V SQL se definuje hned za slovem `SELECT` .
  - Příklad: Chceme vidět jen jméno a e-mail studenta. `sql SELECT jmeno,`  
`email FROM student;`

## D. Typy spojení tabulek (JOINS)

Protože normalizace rozdělí data do mnoha tabulek (např. student a fakulta zvlášť), musíme je při čtení umět propojit přes relace primárních a cizích klíčů. K tomu slouží operace `JOIN` .

Představme si tabulku `Studenti` (vlevo) s `ID_Studenta` , `Jmeno` , `ID_Fakulty` a tabulku `Fakulty` (vpravo) s `ID_Fakulty` , `Nazev_Fakulty` .

- **INNER JOIN (Vnitřní spojení)**: Vrátí pouze ty řádky z obou tabulek, které splňují spojovací podmínku (existuje shoda v obou tabulkách). Pokud student nemá přiřazenou fakultu (nebo fakulta nemá žádného studenta), ve výsledku vůbec nebude. `sql SELECT S.Jmeno, F.Nazev_Fakulty FROM Studenti S INNER`  
`JOIN Fakulty F ON S.ID_Fakulty = F.ID_Fakulty;`
- **LEFT JOIN / LEFT OUTER JOIN (Levé vnější spojení)**: Vrátí všechny řádky z levé tabulky (zde `Studenti` ), a z pravé tabulky ( `Fakulty` ) připojí jen ty řádky, které mají shodu. Pokud pro řádek z levé tabulky neexistuje shoda v pravé, doplní se místo hodnot z pravé tabulky prázdná hodnota `NULL` . (Ideální,

pokud chceme vypsat všechny studenty včetně těch, kteří zatím nemají fakultu). `sql SELECT S.Jmeno, F.Nazev_Fakulty FROM Studenti S LEFT JOIN Fakulty F ON S.ID_Fakulty = F.ID_Fakulty;`

- **RIGHT JOIN / RIGHT OUTER JOIN (Pravé vnější spojení):** Přesný opak `LEFT JOIN`. Vrátí všechny řádky z pravé tabulky (zde `Fakulty`) a z levé (`Studenti`) připojí pouze ty shodné. Pokud pro řádek z pravé tabulky neexistuje shoda v levé, doplní se `NULL`. `sql SELECT S.Jmeno, F.Nazev_Fakulty FROM Studenti S RIGHT JOIN Fakulty F ON S.ID_Fakulty = F.ID_Fakulty;`
- **FULL JOIN / FULL OUTER JOIN (Plné vnější spojení):** Kombinace levého a pravého spojení. Vrátí všechny řádky z obou tabulek. Kde shoda chybí, tam doplní `NULL`. `sql SELECT S.Jmeno, F.Nazev_Fakulty FROM Studenti S FULL JOIN Fakulty F ON S.ID_Fakulty = F.ID_Fakulty;`
- **CROSS JOIN (Kartézský součin):** Spojí každý řádek z první tabulky s každým řádkem z druhé tabulky. Výsledkem je tabulka s počtem řádků rovným součinu počtu řádků obou tabulek. Používá se zřídka, pokud není explicitně potřeba. `sql SELECT S.Jmeno, F.Nazev_Fakulty FROM Studenti S CROSS JOIN Fakulty F;`

## E. Agregční funkce a seskupování

Agregční funkce berou jako vstup celý sloupec hodnot (množinu řádků) a vypočítají z nich jednu jedinou výslednou hodnotu.

- **COUNT()** : Spočítá počet řádků nebo počet ne-NULL hodnot ve sloupci.
  - Příklad: `COUNT(*)` (počet všech řádků), `COUNT(id_studenta)` (počet studentů s ID).
- **SUM()** : Vypočítá sumu (součet) hodnot v číselném sloupci.
  - Příklad: `SUM(kredity)`
- **AVG()** : Vypočítá aritmetický průměr hodnot v číselném sloupci.
  - Příklad: `AVG(znamka)`
- **MIN()** : Najde minimální hodnotu ve sloupci.
  - Příklad: `MIN(datum_narození)`
- **MAX()** : Najde maximální hodnotu ve sloupci.
  - Příklad: `MAX(plat)`

**Klauzule** `GROUP BY` a `HAVING`

Agregace se stává mocnou v kombinaci se seskupováním řádků:

- **GROUP BY** : Rozdělí řádky tabulky do skupin podle hodnot v definovaném sloupci (nebo sloupcích) a agregační funkce pak počítá výsledek pro každou skupinu zvlášť.
  - Příklad: Spočítat počet studentů podle jednotlivých fakult. 

```
sql SELECT ID_Fakulty, COUNT(ID_Studenta) AS PocetStudentu FROM Studenti GROUP BY ID_Fakulty;
```
- **HAVING** : Slouží jako podmínka **WHERE** , ale aplikuje se až na výsledky agregačních funkcí po seskupení. Filtruje skupiny.
  - Příklad: Vypsát jen ty fakulty, které mají více než 500 studentů. 

```
sql SELECT ID_Fakulty, COUNT(ID_Studenta) AS PocetStudentu FROM Studenti GROUP BY ID_Fakulty HAVING COUNT(ID_Studenta) > 500;
```

## F. Množinové operace v SQL

Množinové operace kombinují výsledky dvou samostatných dotazů **SELECT** do jednoho výsledného setu. Oba dotazy musí mít stejný počet sloupců a odpovídající si datové typy v odpovídajících pozicích.

- **UNION (Sjednocení)**: Spojí výsledky obou dotazů dohromady a automaticky odstraní duplicity. 

```
sql SELECT Jmeno FROM Studenti UNION SELECT Jmeno FROM Zamestnanci;
```
- **UNION ALL** : Spojí výsledky obou dotazů dohromady a **zachová i duplicity**. 

```
sql SELECT Jmeno FROM Studenti UNION ALL SELECT Jmeno FROM Zamestnanci;
```
- **INTERSECT (Průnik)**: Vrátí pouze ty řádky, které se vyskytují ve výsledcích obou dotazů současně. 

```
sql SELECT Jmeno FROM Studenti INTERSECT SELECT Jmeno FROM Zamestnanci;
```
- **EXCEPT / MINUS (Rozdíl)**: Vrátí řádky, které jsou ve výsledku prvního dotazu, ale nevyskytují se ve výsledku druhého dotazu. ( **MINUS** je specifické pro Oracle). 

```
sql SELECT Jmeno FROM Studenti EXCEPT SELECT Jmeno FROM Zamestnanci;
```

## G. Vnořené dotazy (Subqueries)

Vnořený dotaz je příkaz **SELECT** umístěný uvnitř jiného SQL příkazu (může být uvnitř jiného **SELECT** , **INSERT** , **UPDATE** nebo **DELETE** ). Vnořený dotaz bývá typicky

uzavřen v kulatých závorkách.

Rozlišujeme dva základní typy:

- **Nekorelovaný vnořený dotaz:** Vnitřní dotaz je zcela nezávislý na vnějším dotazu. Provede se pouze jednou před spuštěním vnějšího dotazu a jeho výsledek (např. jedno číslo, seznam hodnot nebo tabulka) se předá vnějšímu dotazu.
  - Příklad: Najdi studenty, kteří mají lepší průměr než je celkový průměr školy. `sql SELECT jmeno, prumer FROM student WHERE prumer > (SELECT AVG(prumer) FROM student);`
- **Korelovaný vnořený dotaz:** Vnitřní dotaz se odkazuje na hodnoty z aktuálního řádku vnějšího dotazu. To znamená, že vnitřní dotaz se musí vyhodnotit znovu pro každý jednotlivý řádek vnějšího dotazu, což může být u velkých tabulek velmi pomalé. Často se kombinuje s operátory jako `EXISTS` nebo `NOT EXISTS`.
  - Příklad: Najdi studenty, kteří navštěvují alespoň jeden předmět, který má více než 5 kreditů. `sql SELECT S.jmeno FROM Studenti S WHERE EXISTS (SELECT 1 FROM ZapsanePredmety ZP JOIN Predmety P ON ZP.ID_Predmetu = P.ID_Predmetu WHERE ZP.ID_Studenta = S.ID_Studenta AND P.Kredity > 5);`

## H. Indexy, Pohledy, Uložené procedury a funkce, Triggery

Tyto objekty rozšiřují možnosti databáze a zlepšují její výkon, bezpečnost a udržitelnost.

### 1. Indexy

- **Definice:** Speciální vyhledávací struktura, která zlepšuje rychlost operací prohledávání databáze. Funguje podobně jako rejstřík v knize, který ukazuje na konkrétní stránky, kde se nachází hledané slovo.
- **Význam:**
  - **Zrychlení dotazů:** Výrazně zrychlují `SELECT` dotazy, zejména ty s klauzulí `WHERE`, `JOIN` operacemi a `ORDER BY`.
  - **Unikátnost dat:** Unikátní indexy zajišťují, že ve sloupci (nebo kombinaci sloupců) nejsou duplicitní hodnoty.

- **Nevýhody:** Zpomalují `INSERT`, `UPDATE` a `DELETE` operace, protože indexy se musí udržovat aktuální. Zaberou dodatečné místo na disku.
- **Typy indexů:**
  - **Clustered Index (Klastrovaný index):** Určuje fyzické pořadí datových řádků v tabulce. Tabulka může mít pouze jeden klastrovaný index. Často se vytváří automaticky na primárním klíči. Data jsou fyzicky uložena v pořadí indexu.
  - **Non-clustered Index (Neklastrovaný index):** Neovlivňuje fyzické pořadí dat. Obsahuje ukazatele na skutečná data v tabulce. Tabulka může mít více neklastrovaných indexů.
- **Příkazy DDL:**
  - **Vytvoření indexu:** `sql CREATE INDEX idx_student_jmeno ON student (jmeno); CREATE UNIQUE INDEX uidx_student_email ON student (email);`
  - **Smazání indexu:** `sql DROP INDEX idx_student_jmeno ON student; --`  
Syntax se může lišit (např. `DROP INDEX nazev_indexu;` v MySQL)

## 2. Pohledy (Views)

- **Definice:** Virtuální tabulka založená na výsledku dotazu SQL. Neobsahuje data sama o sobě, ale zobrazuje data z jedné nebo více podkladových tabulek v reálném čase.
- **Význam:**
  - **Zjednodušení složitých dotazů:** Umožňuje uložit složitý `JOIN` nebo dotaz s agregacemi pod jednoduchým názvem.
  - **Bezpečnost:** Lze omezit přístup uživatelů pouze na určité sloupce nebo řádky tabulky, aniž by měli přímý přístup k celé tabulce.
  - **Konzistence dat:** Poskytuje konzistentní pohled na data, i když se podkladové schéma mění (pokud je pohled aktualizován).
  - **Modularita:** Zlepšuje modularitu databázového designu.
- **Příkazy DDL:**
  - **Vytvoření pohledu:** `sql CREATE VIEW studenti_praha AS SELECT jmeno, vek, email FROM student WHERE mesto = 'Praha';`
  - **Smazání pohledu:** `sql DROP VIEW studenti_praha;`

## 3. Uložené procedury a funkce (Stored Procedures and Functions)

- **Definice:** Předkompilované bloky SQL příkazů uložené v databázi.

- **Uložená procedura:** Může přijímat parametry a vracet hodnoty (pomocí `OUT` parametrů). Nemusí vracet žádnou hodnotu. Typicky se používá pro provádění akcí (např. vložení, aktualizace dat).
- **Uložená funkce:** Podobná proceduře, ale *musí* vrátit jednu skalární hodnotu (nebo tabulku). Lze ji použít v SQL dotazech jako součást výrazu.
- **Význam:**
  - **Zvýšení výkonu:** Jsou předkompilované, což snižuje režii při opakovaném spouštění.
  - **Modularita a znovupoužitelnost:** Kód je uložen centrálně a může být volán z různých aplikací.
  - **Bezpečnost:** Uživatelům lze udělit oprávnění spouštět procedury/funkce, aniž by měli přímý přístup k podkladovým tabulkám.
  - **Snížení síťového provozu:** Místo odesílání mnoha SQL příkazů se odešle pouze volání procedury/funkce.
- **Příkazy DDL/DML:**
  - **Vytvoření procedury (příklad pro MySQL):** `sql DELIMITER // CREATE PROCEDURE ZvysVekStudenta (IN student_id INT, IN prirustek INT) BEGIN UPDATE student SET vek = vek + prirustek WHERE id = student_id; END // DELIMITER ;`
  - **Volání procedury:** `sql CALL ZvysVekStudenta(1, 1);`
  - **Vytvoření funkce (příklad pro MySQL):** `sql DELIMITER // CREATE FUNCTION GetPocetStudentuFakulty (fakulta_id INT) RETURNS INT DETERMINISTIC BEGIN DECLARE pocet INT; SELECT COUNT(*) INTO pocet FROM Studenti WHERE ID_Fakulty = fakulta_id; RETURN pocet; END // DELIMITER ;`
  - **Použití funkce:** `sql SELECT GetPocetStudentuFakulty(101);`
  - **Smazání procedury/funkce:** `sql DROP PROCEDURE ZvysVekStudenta; DROP FUNCTION GetPocetStudentuFakulty;`

#### 4. Triggery (Spouštěče)

- **Definice:** Speciální typ uložené procedury, která se automaticky spustí (aktivuje) v reakci na určitou událost v databázi (např. `INSERT` , `UPDATE` , `DELETE` na konkrétní tabulce).
- **Význam:**



- **Vynucování komplexních obchodních pravidel:** Lze implementovat složitá pravidla, která nelze vynutit pomocí standardních omezení (CHECK, FOREIGN KEY).
- **Auditování změn:** Zaznamenávání změn dat do auditních tabulek.
- **Udržování referenční integrity:** Implementace kaskádových operací, které nejsou podporovány standardními cizími klíči.
- **Automatická aktualizace souvisejících dat:** Např. aktualizace součtů v jiné tabulce po změně detailních dat.
- **Příkazy DDL:**
  - **Vytvoření triggeru (příklad pro MySQL):**

```
sql DELIMITER // CREATE TRIGGER po_vlozeni_studenta AFTER INSERT ON student FOR EACH ROW BEGIN INSERT INTO log_zmeny (udalost, datum) VALUES ('Nový student vložen', NOW()); END // DELIMITER ;
```
  - **Smazání triggeru:**

```
sql DROP TRIGGER po_vlozeni_studenta;
```

## I. Transakce, ACID vlastnosti, úrovně izolace transakcí

### 1. Transakce

- **Definice:** Logická jednotka práce, která se skládá z jedné nebo více SQL operací (např. `INSERT`, `UPDATE`, `DELETE`). Transakce je navržena tak, aby zajistila integritu dat. Buď se všechny operace v transakci provedou úspěšně (`COMMIT`), nebo se žádná z nich neprovede (`ROLLBACK`).
- **Příkazy pro řízení transakcí:**
  - `BEGIN TRANSACTION;` (nebo `START TRANSACTION;`): Zahájí novou transakci.
  - `COMMIT;`: Potvrdí všechny změny provedené v rámci transakce, čímž se stanou trvalými v databázi.
  - `ROLLBACK;`: Zruší všechny změny provedené v rámci transakce, vrátí databázi do stavu před zahájením transakce.
  - `SAVEPOINT nazev_savepointu;`: Vytvoří bod v transakci, ke kterému se lze vrátit pomocí `ROLLBACK TO nazev_savepointu;`.

### 2. ACID vlastnosti

ACID je akronym popisující sadu vlastností, které zaručují spolehlivost transakcí v databázovém systému.

- **Atomicity (Atomická):**

- **Popis:** Transakce je nedělitelná. Buď se provede celá (všechny její operace), nebo se neprovede vůbec žádná. Neexistuje částečné provedení.
- **Příklad:** Převod peněz z účtu A na účet B. Skládá se z operací "odečíst z A" a "přičíst na B". Pokud jedna selže, obě se vrátí zpět.

- **Consistency (Konzistence):**

- **Popis:** Transakce převede databázi z jednoho konzistentního stavu do jiného konzistentního stavu. Databáze musí dodržovat všechna definovaná pravidla (omezení, integritní pravidla) před zahájením transakce i po jejím dokončení.
- **Příklad:** Po převodu peněz musí součet zůstatků na účtech A a B zůstat stejný (pokud nebyly peníze vytvořeny/zničeny).

- **Isolation (Izolace):**

- **Popis:** Souběžně běžící transakce se navzájem neovlivňují. Každá transakce se jeví, jako by běžela sama, nezávisle na ostatních. Změny jedné transakce nejsou viditelné pro jiné transakce, dokud nejsou potvrzeny.
- **Příklad:** Dvě transakce čtou a zapisují stejná data. Izolace zajistí, že si navzájem nebudou "šlapat na paty" a výsledný stav bude stejný, jako kdyby běžely sériově.

- **Durability (Trvalost):**

- **Popis:** Jakmile je transakce potvrzena ( `COMMIT` ), její změny jsou trvalé a přežijí i selhání systému (např. výpadek napájení, pád serveru). Změny jsou zapsány na trvalé úložiště.
- **Příklad:** Po potvrzení převodu peněz jsou změny zůstatků trvale uloženy a zůstanou zachovány i po restartu databáze.

### 3. Úrovně izolace transakcí

SQL standard definuje čtyři úrovně izolace, které určují, do jaké míry jsou souběžné transakce izolovány jedna od druhé. Nižší úroveň izolace znamená vyšší souběžnost, ale vyšší riziko problémů.

- **Problémy souběžnosti:**

- **Dirty Read (Špinavé čtení):** Transakce A čte data, která byla změněna transakcí B, ale ještě nebyla potvrzena (commitnuta). Pokud se transakce B vrátí zpět ( `ROLLBACK` ), transakce A četla neexistující data.
- **Non-repeatable Read (Neopakovatelné čtení):** Transakce A čte stejný řádek dvakrát a získá různé hodnoty, protože jiná transakce B mezitím data změnila a potvrdila ( `COMMIT` ).
- **Phantom Read (Fantomové čtení):** Transakce A provede dotaz, který vrací sadu řádků. Později stejný dotaz vrátí jinou sadu řádků (více nebo méně), protože jiná transakce B mezitím vložila nebo smazala řádky, které splňují podmínku dotazu.

- **Úrovně izolace (od nejnižší po nejvyšší):**

1. `READ UNCOMMITTED` (**Čtení nepotvrzených dat**):

- **Popis:** Nejnižší úroveň izolace. Transakce může číst nepotvrzené změny jiných transakcí.
- **Problémy:** Povoluje Dirty Read, Non-repeatable Read, Phantom Read.

2. `READ COMMITTED` (**Čtení potvrzených dat**):

- **Popis:** Transakce může číst pouze potvrzené změny.
- **Problémy:** Zabraňuje Dirty Read. Povoluje Non-repeatable Read, Phantom Read.

3. `REPEATABLE READ` (**Opakovatelné čtení**):

- **Popis:** Transakce vidí stejná data při opakovaném čtení stejných řádků.
- **Problémy:** Zabraňuje Dirty Read a Non-repeatable Read. Povoluje Phantom Read.

4. `SERIALIZABLE` (**Serializovatelná**):

- **Popis:** Nejvyšší úroveň izolace. Zajišťuje, že souběžné transakce se chovají, jako by byly prováděny sériově, jedna po druhé.
- **Problémy:** Zabraňuje Dirty Read, Non-repeatable Read i Phantom Read.
- **Nevýhoda:** Nejnižší souběžnost, může vést k výkonostním problémům.

- **Nastavení úrovně izolace:** `sql SET TRANSACTION ISOLATION LEVEL READ COMMITTED;`

## Typické zkouškové otázky

### 1. Vysvětlete pojem CRUD operace a uveďte, jak se mapují na základní SQL příkazy.

- **Odpověď:** CRUD je akronym pro čtyři základní operace: Create (vytvoření), Read (čtení), Update (aktualizace) a Delete (smazání), které představují základní interakci s daty v perzistentních datových strukturách. V SQL se mapují následovně:
  - **Create:** `INSERT`
  - **Read:** `SELECT`
  - **Update:** `UPDATE`
  - **Delete:** `DELETE`

### 2. Popište rozdíl mezi SQL DDL a SQL DML a uveďte příklady příkazů pro každou kategorii.

- **Odpověď:**
  - **SQL DDL (Data Definition Language)** slouží k definici a správě struktury databázových objektů (schématu). Příklady příkazů jsou `CREATE` (např. `CREATE TABLE`), `ALTER` (např. `ALTER TABLE ADD COLUMN`), `DROP` (např. `DROP TABLE`).
  - **SQL DML (Data Manipulation Language)** slouží k manipulaci s daty uvnitř již definovaných databázových struktur. Příklady příkazů jsou `SELECT`, `INSERT`, `UPDATE`, `DELETE`.

### 3. Vysvětlete pojmy selekce a projekce v kontextu relační algebry a SQL dotazů.

- **Odpověď:**
  - **Selekce (restrikce)** je operace relační algebry, která vybírá podmnožinu řádků (n-tic) z tabulky, které splňují zadanou podmínku. V SQL se realizuje pomocí klauzule `WHERE`.
  - **Projekce** je operace relační algebry, která vybírá podmnožinu sloupců (atributů) z tabulky. V SQL se definuje výpisem názvů sloupců za klíčovým slovem `SELECT`.

4. **Popište alespoň tři různé typy JOIN operací v SQL a vysvětlete, jak se liší ve způsobu kombinování dat z tabulek.**

○ **Odpověď:**

- **INNER JOIN** : Vrátí pouze ty řádky, pro které existuje shoda ve spojovacím sloupci v obou tabulkách. Řádky bez shody jsou ignorovány.
- **LEFT JOIN (nebo LEFT OUTER JOIN)** : Vrátí všechny řádky z levé tabulky a odpovídající řádky z pravé tabulky. Pokud v pravé tabulce není shoda, hodnoty z pravé tabulky budou **NULL** .
- **RIGHT JOIN (nebo RIGHT OUTER JOIN)** : Vrátí všechny řádky z pravé tabulky a odpovídající řádky z levé tabulky. Pokud v levé tabulce není shoda, hodnoty z levé tabulky budou **NULL** .
- **FULL JOIN (nebo FULL OUTER JOIN)** : Vrátí všechny řádky z obou tabulek. Pokud v jedné tabulce není shoda, hodnoty z chybějící tabulky budou **NULL** .

5. **Co jsou agregační funkce a jak se používají klauzule GROUP BY a HAVING ? Uveďte příklad.**

○ **Odpověď:** Agregační funkce (např. **COUNT()** , **SUM()** , **AVG()** , **MIN()** , **MAX()** ) zpracovávají skupinu řádků a vrací jednu souhrnnou hodnotu.

- **GROUP BY** klauzule seskupuje řádky s identickými hodnotami v jednom nebo více sloupcích do souhrnných řádků. Agregační funkce se pak aplikují na každou skupinu zvlášť.
- **HAVING** klauzule se používá k filtrování skupin vytvořených pomocí **GROUP BY** na základě podmínek, které se vztahují na výsledky agregačních funkcí.
- **Příklad:** `SELECT ID_Fakulty, COUNT(ID_Studenta) FROM Studenti GROUP BY ID_Fakulty HAVING COUNT(ID_Studenta) > 100;` (Spočítá studenty na fakultách a vybere jen fakulty s více než 100 studenty).

6. **Vysvětlete pojem transakce a popište ACID vlastnosti.**

- **Odpověď:** Transakce je logická jednotka práce, která se skládá z jedné nebo více SQL operací. Buď se všechny operace provedou úspěšně

( `COMMIT` ), nebo se žádná z nich neprovede ( `ROLLBACK` ). ACID je akronym pro vlastnosti, které zajišťují spolehlivost transakcí:

- **Atomicity:** Transakce je nedělitelná; buď se provede celá, nebo vůbec.
- **Consistency:** Transakce převede databázi z jednoho konzistentního stavu do jiného konzistentního stavu.
- **Isolation:** Souběžné transakce se navzájem neovlivňují; každá se jeví, jako by běžela sama.
- **Durability:** Jakmile je transakce potvrzena, její změny jsou trvalé a přežijí i selhání systému.

## 7. Popište, co jsou indexy, pohledy a trigger v databázi a jaký je jejich význam.

### ○ Odpověď:

- **Indexy:** Jsou speciální vyhledávací struktury, které zrychlují operace čtení dat (např. `SELECT` , `S` , `WHERE` , `JOIN` , `ORDER BY` ) tím, že poskytují rychlý přístup k datům. Fungují jako rejstřík v knize.
  - **Pohledy (Views):** Jsou virtuální tabulky založené na výsledku SQL dotazu. Neobsahují data, ale poskytují zjednodušený, bezpečný a konzistentní pohled na podkladová data.
  - **Triggery:** Jsou speciální typy uložených procedur, které se automaticky spustí v reakci na určitou událost (např. `INSERT` , `UPDATE` , `DELETE` ) na konkrétní tabulce. Používají se pro vynucování komplexních pravidel, auditování nebo automatickou aktualizaci dat.
-

## 31. Spouště a uložené procedury

### Klíčové pojmy

- **Uložená procedura (Stored Procedure):** Pojmenovaný blok SQL kódu uložený a zkompilovaný v databázi, který lze explicitně volat.
- **Uložená funkce (Stored Function):** Podobná proceduře, ale musí vracet skalární hodnotu a může být použita v SQL výrazech.
- **Spoušť (Trigger):** Speciální typ uložené procedury, která se automaticky aktivuje jako reakce na definovanou DML událost (INSERT, UPDATE, DELETE) na tabulce.
- **Audit:** Záznam změn dat v databázi, často realizovaný pomocí spouští.
- **BEFORE :** Časování spouště, kód se provede před samotnou DML operací.
- **AFTER :** Časování spouště, kód se provede po úspěšném dokončení DML operace.
- **IN parametr:** Vstupní parametr procedury, hodnota se předává dovnitř.
- **OUT parametr:** Výstupní parametr procedury, procedura do něj zapíše výsledek.
- **INOUT parametr:** Kombinovaný parametr, hodnota vstoupí, procedura ji modifikuje a vrátí upravenou.
- **OLD / NEW reference:** Pseudotabulární reference ve spouštích pro přístup k původním a novým hodnotám řádku.

---

### A. Uložené procedury (Stored Procedures)

Uložená procedura je pojmenovaný blok SQL kódu, který je zkompilován a uložen přímo v databázi. Může obsahovat deklarace proměnných, podmínky `IF`, cykly `WHILE` a vestavěné SQL příkazy.

**Volání:** Spouští se ručně z aplikace nebo z databázové konzole pomocí příkazu `CALL nazev_procedury(parametry);` .

**Parametry:** Procedury mohou přijímat a vracet data pomocí tří typů parametrů:

- **IN (vstupní):**
  - Hodnota předaná z aplikace dovnitř procedury.
  - Je to výchozí typ parametru.
  - Příklad: `CREATE PROCEDURE GetStudentName (IN student_id INT)`
- **OUT (výstupní):**
  - Procedura do tohoto parametru zapíše výsledek, který si pak aplikace po skončení procedury přečte.
  - Příklad: `CREATE PROCEDURE GetStudentCount (OUT total_students INT)`
- **INOUT (kombinovaný):**
  - Proměnná vstoupí s hodnotou, procedura ji uvnitř modifikuje a vrátí upravenou.
  - Příklad: `CREATE PROCEDURE UpdateCounter (INOUT current_value INT)`

**Návratová hodnota:** Procedura nemusí vracet žádnou hodnotu (funguje jako procedura v programování), ale může jako svůj výsledek vrátit celou výslednou množinu řádků (Result Set) pomocí příkazu `SELECT`. Uložené funkce (Stored Functions) jsou speciální typ procedur, které *musí* vracet jednu skalární hodnotu a mohou být použity přímo v SQL výrazech (např. v `SELECT` klauzuli).

### Výhody uložených procedur:

- **Snížení síťového provozu:** Pokud aplikace potřebuje provést komplexní operaci sestávající z desítek dotazů, nemusí je posílat po síti jeden po druhém. Pošle pouze jeden příkaz `CALL`, celá logika proběhne lokálně na serveru a aplikaci se vrátí až finální výsledek.
- **Vyšší výkon:** Databáze si proceduru při prvním uložení zkompiluje a vytvoří pro ni optimální prováděcí plán, který pak opakovaně využívá. To snižuje režii spojenou s parsováním a optimalizací dotazů při každém spuštění.
- **Zvýšení bezpečnosti:** Uživatelům nebo aplikaci lze odebrat přímá práva ke čtení a zápisu do tabulek (příkazy `SELECT`, `INSERT`, `UPDATE`, `DELETE`), a namísto toho jim povolit pouze spouštění konkrétních procedur (např. `CALL registruj_studenta()`). Tím se zabrání neautorizovaným úpravám a útokům typu SQL Injection, pokud jsou procedury správně navrženy.



- **Centralizace logiky:** Obchodní logika může být uložena přímo v databázi, což zajišťuje její konzistentní aplikaci napříč různými aplikacemi a klienty.
- **Modularita a znovupoužitelnost:** Komplexní operace lze rozdělit do menších, spravovatelných procedur, které lze opakovaně používat.

### Příklad uložené procedury (MySQL syntaxe):

```
DELIMITER //
```

```
CREATE PROCEDURE GetStudentInfo (IN student_id INT, OUT student_name VARCHAR(100))  
BEGIN  
    SELECT jmeno INTO student_name FROM student WHERE id = student_id;  
END //
```

```
DELIMITER ;
```

```
-- Volání procedury  
CALL GetStudentInfo(1, @name);  
SELECT @name;
```

## B. Spouště (Triggery)

Spoušť (Trigger) je speciální typ uložené procedury, kterou na rozdíl od běžné procedury nelze volat ručně. Spoušť je pevně navázána na konkrétní tabulku a automaticky se aktivuje (spustí) jako reakce na definovanou událost DML (Data Manipulation Language): `INSERT` , `UPDATE` nebo `DELETE` .

**Specifikace spouště:** Při definici spouště musíme určit tři základní věci:

- **Událost (Event):**
  - Nad jakou operací má spoušť bdít:
    - `INSERT`
    - `UPDATE`
    - `DELETE`
- **Časování (Timing):**
  - Kdy přesně se má kód spustit vzhledem k DML události:

- **BEFORE** : Kód se provede *před* samotným zapsáním/přepsáním/smazáním řádku. Používá se typicky pro validaci a úpravu dat (např. kontrola, zda zadaný věk není záporný, nebo automatické převedení e-mailu na malá písmena před uložením). Může zabránit provedení operace.
- **AFTER** : Kód se provede až *po* úspěšném dokončení operace nad tabulkou. Používá se pro logování změn, auditování nebo kaskádové aktualizace v jiných tabulkách.

- **Úroveň zpracování:**

- **FOR EACH ROW (Řádkový trigger):** Kód se spustí nezávisle pro každý ovlivněný řádek. Pokud jedním příkazem `UPDATE` upravíte 100 studentů, řádkový trigger se spustí stokrát.
- **FOR EACH STATEMENT (Příkazový trigger):** Kód se spustí pouze jednou pro celý SQL příkaz, bez ohledu na to, kolik řádků reálně ovlivnil. (Tento typ není podporován ve všech databázích, např. MySQL podporuje pouze řádkové trigger).

**Speciální proměnné ve spouštích:** Uvnitř řádkových spouští máme přístup k původnímu a nově vkládanému stavu dat pomocí pseudotabulárních referencí:

- **OLD** : Reprezentuje hodnoty řádku *před* úpravou (přístupné v `UPDATE` a `DELETE` triggerech).
- **NEW** : Reprezentuje hodnoty řádku, které se *budou zapisovat* (přístupné v `INSERT` a `UPDATE` triggerech).
  - Příklad použití: `IF NEW.vek < 0 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Věk nemůže být záporný'; END IF;`

**Typické použití spouští:**

- **Vynucování komplexních integritních omezení:** Kontrola pravidel, která nelze vyjádřit pomocí standardních `CHECK` podmínek (např. kontrola stavu konta před schválením platby, aby nedošlo k přečerpání).
- **Auditování a logování:** Automatické zapisování historie změn do speciální auditní tabulky (kdo, kdy a co v databázi změnil). To je klíčové pro sledování datových změn a dodržování předpisů.

- **Generování odvozených hodnot:** Automatický dopočet statistik, skladových zásob nebo jiných odvozených hodnot v jiných tabulkách při přidání, úpravě nebo smazání souvisejících dat.
- **Kaskádové operace:** Provádění složitějších kaskádových akcí, než jaké nabízejí referenční omezení ( `ON DELETE CASCADE` ).

### Příklad spouště (MySQL syntaxe):

```
DELIMITER //
```

```
CREATE TRIGGER audit_student_update
AFTER UPDATE ON student
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (entity, entity_id, action, old_value, new_value, char
VALUES ('student', OLD.id, 'UPDATE', OLD.jmeno, NEW.jmeno, USER(), NOW());
END //
```

```
DELIMITER ;
```

## C. Porovnání: Kdy použít Proceduru a kdy Trigger?

| Vlastnost     | Uložená procedura (Stored Procedure)                                                           |
|---------------|------------------------------------------------------------------------------------------------|
| Aktivace      | Explicitní volání ( <code>CALL</code> ) z aplikace nebo jiné procedury.                        |
| Účel          | Provádění komplexních úloh, obchodní logika, transakce, \                                      |
| Závislost     | Nezávislá na konkrétní tabulce (může pracovat s více tabu                                      |
| Vstup/Výstup  | Může mít <code>IN</code> , <code>OUT</code> , <code>INOUT</code> parametry, vrací výsledné sad |
| Kontrola toku | Plná kontrola nad spuštěním a tokem logiky.                                                    |
| Viditelnost   | Její existence je zřejmá z kódu aplikace, která ji volá.                                       |

| Vlastnost          | Uložená procedura (Stored Procedure)                       |
|--------------------|------------------------------------------------------------|
| Chyby              | Chyby se obvykle projeví v aplikaci, která proceduru volá. |
| Transakční kontext | Může definovat vlastní transakce.                          |

#### Kdy použít uloženou proceduru:

- Když potřebujete provést složitou obchodní logiku, která zahrnuje více SQL příkazů a je volána na vyžádání.
- Pro operace, které vyžadují vstupní parametry a vracejí specifické výsledky (skalární hodnoty nebo sady řádků).
- Pro zvýšení bezpečnosti a abstrakce databázových operací od aplikační vrstvy.
- Pro zlepšení výkonu opakovaně prováděných komplexních dotazů.

#### Kdy použít spoušť:

- Když potřebujete automaticky vynucovat komplexní integritní omezení, která nelze řešit standardními `CHECK` nebo `FOREIGN KEY` omezeními.
- Pro automatické auditování a logování změn dat.
- Pro udržování konzistence dat napříč tabulkami, kde je potřeba kaskádově aktualizovat nebo vkládat data v reakci na změny v jiné tabulce.
- Když je potřeba, aby se určitá logika provedla *vždy* při konkrétní DML operaci, bez ohledu na to, jakým způsobem byla operace vyvolána.

---

#### Praktické použití

- **Audit změn:** Automatické zaznamenávání všech úprav dat pro bezpečnostní nebo regulační účely (triggery).
- **Složitě databázové operace:** Provádění komplexních transakcí, které vyžadují koordinaci více kroků (uložené procedury).
- **Validace pravidel:** Kontrola dat před jejich uložením do databáze, aby se zajistila jejich správnost a konzistence (triggery, uložené procedury).
- **Dávkové zpracování:** Efektivní zpracování velkého množství dat v jedné operaci (uložené procedury).

- **Zabezpečení dat:** Omezení přímého přístupu k tabulkám a vynucení interakce přes definované rozhraní (uložené procedury).
  - **Generování ID nebo výchozích hodnot:** Automatické přiřazení hodnot sloupcům (triggery).
- 

## Nejčastější chyby u zkoušky

- **Nadměrné ukládání aplikační logiky do triggerů:** Triggery by měly být co nejjednodušší a zaměřené na integritu dat. Složitá obchodní logika patří spíše do uložených procedur nebo aplikační vrstvy.
  - **Neřešení transakčního kontextu:** Triggery se spouští v rámci transakce, která vyvolala DML operaci. Chyba v triggeru může způsobit rollback celé transakce. U procedur je třeba transakce explicitně řídit.
  - **Těžko dohledatelné automatické změny dat:** Triggery mohou provádět změny dat "na pozadí", což může být pro vývojáře a administrátory matoucí a ztěžovat ladění. Je nutná dobrá dokumentace.
  - **Předpoklad, že trigger jsou vždy řádkové:** Některé databáze podporují i příkazové trigger, což mění jejich chování.
  - **Ignorování výkonnostních dopadů triggerů:** Složité trigger mohou výrazně zpomalit DML operace.
- 

## Typické zkouškové otázky

### 1. Co je trigger a kdy se spouští?

- **Odpověď:** Trigger (spoušť) je speciální typ uloženého kódu v databázi, který se automaticky aktivuje (spouští) jako reakce na specifickou DML (Data Manipulation Language) událost ( `INSERT` , `UPDATE` , `DELETE` ) na konkrétní tabulce. Spouští se buď `BEFORE` (před provedením DML operace) nebo `AFTER` (po provedení DML operace), a to buď `FOR EACH ROW` (pro každý ovlivněný řádek) nebo `FOR EACH STATEMENT` (jednou pro celý příkaz).

### 2. Jak se liší trigger a stored procedure?

- **Odpověď:** Hlavní rozdíl spočívá v jejich aktivaci a účelu.

- **Uložená procedura** je explicitně volána uživatelem nebo aplikací pomocí příkazu `CALL`. Slouží k provádění komplexní obchodní logiky, transakcí a vracení dat. Může mít `IN`, `OUT`, `INOUT` parametry a pracovat s více tabulkami.
- **Trigger** se aktivuje automaticky v reakci na DML událost na konkrétní tabulce a nelze jej volat ručně. Jeho primárním účelem je vynucování integritních omezení, auditování, logování a kaskádové akce. Pracuje s pseudoproměnnými `OLD` a `NEW` pro přístup k datům ovlivněného řádku.

### 3. Jaké jsou výhody a nevýhody databázové logiky (procedur a triggerů)?

#### ○ Odpověď:

##### ■ Výhody:

- **Zvýšený výkon:** Kód je zkompilován a optimalizován databází, snižuje se síťový provoz.
- **Zvýšená bezpečnost:** Lze omezit přímý přístup k tabulkám a povolit pouze spouštění procedur.
- **Centralizace logiky:** Zajišťuje konzistentní aplikaci obchodních pravidel napříč různými aplikacemi.
- **Integrita dat:** Triggery automaticky vynucují komplexní pravidla pro udržení správnosti dat.
- **Modularita a znovupoužitelnost:** Kód je uspořádán a lze jej opakovaně používat.

##### ■ Nevýhody:

- **Závislost na konkrétní databázi:** Kód je často specifický pro daný databázový systém, což ztěžuje migraci.
- **Obtížnější ladění a testování:** Logika je oddělena od aplikačního kódu, což může komplikovat ladění, zejména u triggerů, které se spouští na pozadí.
- **Skryté chování:** Triggery mohou provádět změny, které nejsou zřejmé z aplikačního kódu, což může vést k neočekávaným výsledkům a ztížit údržbu.
- **Zvýšená zátěž databáze:** Složitá logika v databázi může zatížit databázový server.
- **Těžko dohledatelné automatické změny dat:** Změny provedené triggery mohou být obtížně sledovatelné bez

dobré dokumentace.

---

## **Shrnutí kapitoly**

Triggery a uložené procedury jsou mocné nástroje, které přesouvají část aplikační logiky přímo do databáze. Uložené procedury nabízejí výhody v podobě snížení síťového provozu, zvýšení výkonu a bezpečnosti díky centralizaci a optimalizaci kódu. Triggery se automaticky spouští v reakci na DML události a jsou ideální pro vynucování komplexních integritních omezení, auditování a kaskádové operace. Oba mechanismy jsou silné, ale vyžadují pečlivý návrh, dobrou dokumentaci a testování, aby se předešlo problémům s laděním, výkonem a neočekávaným chováním.

Tento studijní materiál je připraven s ohledem na komplexní pokrytí okruhu SZZ, doplněný o detaily a přesné formulace dle standardů počítačových věd.

---

## 32. Pohledy, přístupová práva.

### Transakce, princip, vlastnosti transakcí.

#### A. Pohledy (Views)

Pohled (View) je **virtuální tabulka**, která nemá svá vlastní data fyzicky uložená na disku. Je definována pomocí uloženého SQL dotazu `SELECT`. Když se aplikace dotáže na pohled, databáze na pozadí transparentně vykoná tento definovaný dotaz nad reálnými (podkladovými) tabulkami.

#### Vytvoření a odstranění pohledu

Pohled se vytváří pomocí příkazu `CREATE VIEW` :

```
CREATE VIEW v_student_info AS
SELECT
    s.id,
    s.jmeno,
    s.prijmeni,
    k.nazev AS obor
FROM
    student s
JOIN
    katedra k ON s.katedra_id = k.id
WHERE
    s.aktivni = TRUE;
```

Pohled se odstraní příkazem `DROP VIEW` :

```
DROP VIEW v_student_info;
```



## Výhody a využití pohledů

- **Zjednodušení komplexních dotazů:**

- Pokud aplikace často potřebuje spojovat více tabulek přes složité `JOIN` podmínky a agregovat data, lze toto spojení zapouzdřit do jednoho pohledu.
- Vývojář pak píše pouze jednoduchý dotaz, např.: `SELECT`
  - `FROM v_statistika_studentu;` .

- **Zvýšení bezpečnosti (Abstrakce dat):**

- Pohledy umožňují skrýt citlivá data.
- Místo zpřístupnění celé tabulky `Zamestnanec` (která obsahuje i platy a rodná čísla) vytvoříme pohled `v_verejny_seznam_zamestnancu` , který obsahuje pouze jméno, oddělení a e-mail.
- Uživatelům pak udělíme přístup pouze k tomuto pohledu.

- **Logická nezávislost dat:**

- Pokud se v budoucnu změní vnitřní struktura databáze (např. jedna tabulka se rozdělí na dvě kvůli normalizaci), stačí upravit definici pohledu.
- Kód v klientské aplikaci, který z pohledu čte, se nemusí vůbec měnit.

- **Updatovatelné pohledy:**

- Některé pohledy, které jsou založeny na jedné tabulce a neobsahují agregace nebo složité `JOIN` y, mohou být updatovatelné.
- To znamená, že příkazy `INSERT` , `UPDATE` , `DELETE` provedené nad pohledem se promítnou do podkladové tabulky.
- Většina komplexních pohledů je však pouze pro čtení.

## Materiálové (Materializované) pohledy

- Na rozdíl od běžných pohledů mají materializované pohledy data **reálně zkopírovaná a uložená na disku**.
- Používají se v datových skladech k dramatickému zrychlení obřích analytických dotazů, protože dotaz se provádí nad předpočítanými daty,

nikoli nad živými tabulkami.

- Jejich nevýhodou je, že se musí periodicky aktualizovat (např. pomocí `REFRESH MATERIALIZED VIEW` ), takže data v nich nemusí být v reálném čase stoprocentně aktuální.

## B. Přístupová práva

Databázový systém (DBMS) musí garantovat, že k datům mají přístup pouze autorizovaní uživatelé a aplikace. K řízení přístupu se v SQL používají příkazy spadající do podjazyka **DCL (Data Control Language)**.

### Uživatelé a role

- **Uživatel (User):** Konkrétní entita, která se připojuje k databázi (např. `tomas` , `aplikace_web` ).
- **Role (Role):** Skupina oprávnění, která se přiděluje uživatelům. Zjednodušuje správu práv, protože práva se přidělují rolím a uživatelé se pak pouze přiřazují k rolím. To je základ **RBAC (Role-Based Access Control)**.

### Příkazy DCL

- **GRANT (Udělení práv):**
  - Umožňuje uživateli nebo roli provádět konkrétní operace nad konkrétním databázovým objektem (tabulka, pohled, procedura).
  - **Příklad:** Udělení práv pro čtení a vkládání do tabulky `student` pro roli `sekretariat` . `sql GRANT SELECT, INSERT ON student TO 'sekretariat';`
  - **Příklad:** Udělení všech práv na tabulku `kurzy` pro uživatele `admin_ucitel` s možností předávat tato práva dál. `sql GRANT ALL PRIVILEGES ON kurzy TO 'admin_ucitel' WITH GRANT OPTION;`
  - Běžná oprávnění zahrnují: `SELECT` , `INSERT` , `UPDATE` , `DELETE` , `REFERENCES` (pro cizí klíče), `EXECUTE` (pro procedury/funkce), `CREATE` (pro vytváření objektů).
- **REVOKE (Odebrání práv):**
  - Odstraní dříve udělená práva od uživatele nebo role.

- **Příklad:** Odebrání práva na vkládání do tabulky `student` od role `sekretariat`.  
`sql REVOKE INSERT ON student FROM 'sekretariat';`

## C. Transakce a jejich vlastnosti (ACID)

Transakce je **nedělitelná posloupnost jednoho nebo více SQL příkazů (DML)**, která tvoří jednu logickou jednotku práce s databází. Typickým příkladem je bankovní převod: peníze se musí z účtu A odečíst ( `UPDATE` ) a na účet B připsat ( `UPDATE` ). Pokud by se provedl jen první příkaz a systém havaroval, peníze by zmizely. Transakce garantuje, že se provede buď vše, nebo nic.

### Řízení transakcí v SQL

- **START TRANSACTION (nebo BEGIN):**
  - Označuje začátek transakce. Všechny následující úpravy jsou dočasné a viditelné pouze pro aktuální relaci.
  - **Příklad:** `sql BEGIN; UPDATE account SET balance = balance - 100 WHERE id = 1; UPDATE account SET balance = balance + 100 WHERE id = 2; COMMIT;`
- **COMMIT (Potvrzení):**
  - Úspěšné dokončení transakce. Všechny změny provedené v rámci transakce se trvale zapíší na disk a stanou se viditelnými pro ostatní uživatele.
- **ROLLBACK (Odvolání / Návrat):**
  - Transakce selhala (např. kvůli chybě, porušení integrity nebo rozhodnutí uživatele).
  - Všechny dočasné změny provedené od spuštění transakce se kompletně stornují a databáze se vrátí do přesného stavu, v jakém byla před spuštěním transakce.
- **SAVEPOINT (Uložení bodu):**
  - Umožňuje definovat bod v transakci, ke kterému se lze vrátit pomocí `ROLLBACK TO SAVEPOINT`.

- To je užitečné pro částečné stornování operací v rámci jedné delší transakce.

## Vlastnosti transakcí: Koncept ACID

Každý transakční databázový systém musí striktně splňovat čtyři základní vlastnosti označované zkratkou **ACID**:

- **Atomarita (Atomicity - Nedělitelnost):**

- Transakce se chová jako jediný atomický krok.
- Buď se úspěšně vykonají všechny příkazy uvnitř transakce ( `COMMIT` ), nebo se v případě chyby či pádu systému nevykoná žádný ( `ROLLBACK` ).
- Databáze nesmí nikdy zůstat v napůl rozpracovaném stavu.

- **Konzistence (Consistency - Soudržnost):**

- Transakce musí převést databázi z jednoho validního (konzistentního) stavu do jiného validního stavu.
- Během přechodu nesmí být porušena žádná integritní omezení (cizí klíče, unikátní indexy, `CHECK` podmínky, doménová integrita).
- Pokud by transakce integritu porušila, je okamžitě stornována.

- **Izolovanost (Isolation):**

- Pokud v databázi běží více transakcí paralelně vedle sebe, nesmí se navzájem ovlivňovat ani vidět své dočasné, dosud nepotvrzené změny.
- Navenek se systém musí chovat tak, jako by transakce běžely sekvenčně jedna po druhé.
- **Izolační úrovně (Anomálie):** Podle nastavení izolační úrovně (např. `READ UNCOMMITTED` , `READ COMMITTED` , `REPEATABLE READ` , `SERIALIZABLE` ) databáze řeší anomálie jako:
  - **Dirty Read:** Čtení nepotvrzených dat jiné transakce.
  - **Non-repeatable Read:** Změna dat pod rukama během jedné transakce (stejný dotaz vrátí různé výsledky).
  - **Phantom Read:** Nové řádky se objeví v rozsahu dat, která byla přečtena během jedné transakce.
- K zajištění izolovanosti databáze interně využívají mechanismy jako zamykání řádků/tabulek (locking) nebo multiverzní kontrolu

souběžnosti (MVCC - Multi-Version Concurrency Control).

- **Trvalost (Durability - Stálost):**

- Jakmile transakce úspěšně projde přes `COMMIT`, její změny jsou trvale zapsány na disk do perzistentního úložiště.
- Tyto změny již nesmí být ztraceny ani v případě okamžitého výpadku napájení nebo totálního pádu operačního systému hned v následující milisekundě.
- Databáze toho dosahuje zápisem do transakčního logu (WAL - Write-Ahead Logging) ještě před samotnou úpravou datových stránek.

---

## Typické zkouškové otázky

### 1. Co je view a k čemu slouží?

- **Odpověď:** Pohled (View) je virtuální tabulka, která nemá fyzicky uložená data, ale je definována uloženým SQL dotazem `SELECT`. Slouží k:
  - **Zjednodušení komplexních dotazů:** Zapouzdřuje složité `JOIN` y a agregace do jednoduchého dotazu.
  - **Zvýšení bezpečnosti:** Umožňuje skrýt citlivá data a zpřístupnit uživatelům pouze relevantní podmnožinu dat.
  - **Logická nezávislost dat:** Poskytuje abstrakci nad podkladovou strukturou tabulek, takže změny v tabulkách nemusí ovlivnit klientské aplikace.

### 2. Vysvětlete ACID.

- **Odpověď:** ACID je akronym pro čtyři základní vlastnosti, které musí splňovat transakční databázový systém, aby zajistil spolehlivost zpracování dat:
  - **Atomarita (Atomicity):** Transakce je nedělitelná jednotka práce. Buď se všechny její operace provedou úspěšně, nebo se žádná neprovede (vše se vrátí do původního stavu).
  - **Konzistence (Consistency):** Transakce musí převést databázi z jednoho platného stavu do jiného platného stavu. Nesmí porušit

žádná integritní omezení databáze.

- **Izolovanost (Isolation):** Souběžně běžící transakce se nesmí navzájem ovlivňovat. Každá transakce musí mít dojem, že běží v systému sama.
- **Trvalost (Durability):** Jakmile je transakce potvrzena ( `COMMIT` ), její změny jsou trvale uloženy a přežijí i případné selhání systému (např. výpadek napájení).

### 3. Jak funguje COMMIT a ROLLBACK?

#### ○ Odpověď:

- **COMMIT :** Příkaz `COMMIT` slouží k trvalému uložení všech změn provedených v rámci aktuální transakce do databáze. Jakmile je transakce potvrzena, změny se stanou viditelnými pro ostatní uživatele a jsou garantovány jako trvalé (vlastnost Durability).
- **ROLLBACK :** Příkaz `ROLLBACK` slouží ke zrušení všech změn provedených v rámci aktuální transakce. Databáze se vrátí do stavu, v jakém byla před zahájením transakce. To se používá v případě chyby, nekonzistence nebo pokud se transakce z jakéhokoli důvodu nemůže dokončit.

Níže naleznete studijní materiál k okruhu "33. Indexování a optimalizace dotazů", doplněný a naformátovaný dle vašich přísných pravidel.

---

## 33. Indexování a optimalizace dotazů

### Klíčové pojmy

- **Index:** Pomocná datová struktura pro zrychlení vyhledávání.
  - **Full Table Scan:** Prohledávání celé tabulky.
  - **B-strom (B+ Tree):** Vyvážená stromová struktura, nejčastější typ indexu.
  - **Hašovací index (Hash Index):** Index založený na hašovací funkci, rychlý pro přesné shody.
  - **Clustered Index (Primární index):** Fyzicky určuje pořadí řádků na disku, tabulka může mít jen jeden.
  - **Non-Clustered Index (Sekundární index):** Uložen odděleně, odkazuje na řádky v tabulce (nebo clustered indexu).
  - **Composite Index (Složený index):** Index nad více sloupci.
  - **Query Optimizer (Optimalizátor dotazů):** Komponenta databáze, která vybírá nejefektivnější prováděcí plán.
  - **Execution Plan (Prováděcí plán):** Sekvence operací, kterou databáze zvolí pro vykonání dotazu.
  - **EXPLAIN:** SQL příkaz pro zobrazení prováděcího plánu.
  - **Covering Index:** Index, který obsahuje všechny sloupce potřebné pro dotaz, takže není nutné sahat do hlavní tabulky.
- 

### A. Co je to index a k čemu slouží?

Představte si tlustou, tisícistránkovou tištěnou knihu o programování a úkol najít v ní kapitolu o "vláknech". Pokud kniha nemá žádný rejstřík, musíte začít od první stránky a listovat slovo od slova až na konec. V databázích se tomuto přístupu říká **Full Table Scan** (celotabulkový scan) – databáze musí načíst z disku do paměti RAM úplně všechny řádky tabulky a každý zkontrolovat. U tabulky s miliony záznamů je to extrémně pomalé.

**Index** je pomocná datová struktura vytvořená nad konkrétním sloupcem (nebo skupinou sloupců) tabulky, která funguje přesně jako abecední rejstřík na konci knihy. Obsahuje seřazené hodnoty z daného sloupce a ukazatele (adresy) na reálné řádky v datové tabulce, kde se tyto hodnoty nacházejí.

- **Hlavní výhoda:** Dramatické zrychlení vyhledávání ( `SELECT` s podmínkou `WHERE` ), řazení ( `ORDER BY` ) a propojování tabulek ( `JOIN` ).
- **Nevýhody (Cena za index):**
  - Indexy zabírají dodatečné místo na disku a v paměti RAM.
  - Zpomalují operace zápisu ( `INSERT` , `UPDATE` , `DELETE` ). Kdykoliv se totiž data v tabulce změní, databázový engine musí automaticky přepočítat a aktualizovat i všechny indexy, které nad touto tabulkou visí.

## B. Vnitřní struktury indexů

U státní zkoušky je potřeba vědět, jak jsou indexy fyzicky implementovány.

Nejčastěji se setkáte se dvěma strukturami:

1. **B-Strom (Balanced Tree / B+Tree)** Jedná se o nejrozšířenější typ indexu, který je výchozím v naprosté většině relačních databází (např. InnoDB v MySQL).
  - **Struktura:** Jde o vyvážený strom. V kořenovém uzlu a vnitřních uzlech jsou uloženy rozsahy hodnot, které navigují vyhledávání směrem dolů. V nejnižší vrstvě (tzv. listech / leaf nodes) jsou pak uloženy samotné klíče a ukazatele na reálná data na disku. V případě B+stromu jsou všechny listové uzly navíc propojeny spojovým seznamem pro efektivní procházení rozsahu.
  - **Proč je vyvážený:** Cesta od kořene k jakémukoliv listu je vždy stejně dlouhá. Vyhledávání má logaritmickou časovou složitost  $O(\log n)$ , kde  $n$  je počet záznamů.
  - **Použití:** Skvělý pro přesné shody ( `=` ), ale hlavně pro rozsahové dotazy ( `BETWEEN` , `>` , `<` ), protože listy stromu jsou navíc lineárně propojeny spojovým seznamem.

### 2. Hašovací index (Hash Index)



- **Struktura:** Využívá hašovací funkci, která vezme hodnotu sloupce a transformuje ji na konkrétní adresu v paměti (tzv. bucket).
- **Složitost:** V ideálním případě dosahuje konstantní časové složitosti  $O(1)$  – vyhledání je okamžité bez ohledu na velikost tabulky.
- **Omezení:** Funguje pouze pro přesnou shodu ( `=` , `IN` ). Je naprosto nepoužitelný pro rozsahové dotazy ( `>` , `<` ) nebo řazení, protože hašování kompletně zničí informaci o uspořádání dat.

---

## C. Typy indexů z hlediska logiky

- **Primární index (Clustered Index):**

- Fyzicky určuje pořadí, v jakém jsou řádky tabulky reálně zapsány na disku.
- Listy B-stromu v tomto případě neobsahují jen ukazatele, ale přímo reálná data řádků.
- Každá tabulka může mít maximálně jeden clustered index (standardně se vytváří automaticky nad primárním klíčem).

- **Sekundární index (Non-Clustered Index):**

- Je uložen odděleně od samotné tabulky. V listech stromu má uloženou hodnotu klíče a ukazatel na příslušný řádek v clustered indexu (nebo přímo na fyzickou adresu řádku, pokud clustered index neexistuje).
- Nad tabulkou jich můžete vytvořit libovolné množství (např. nad sloupcem `prijmeni` , `email` ).

- **Složený index (Composite Index):**

- Index vytvořený nad více sloupci současně (např. `(prijmeni, jmeno)` ).
  - U složených indexů záleží na pořadí sloupců – index lze využít i tehdy, pokud v podmínce `WHERE` hledáte pouze podle prvního sloupce, ale nelze ho použít, pokud hledáte pouze podle druhého (např. index na `(A, B)` pomůže pro `WHERE A = 'x'` , ale ne pro `WHERE B = 'y'` ).
-

## D. Optimalizace dotazů a prováděcí plán (Execution Plan)

Když do databáze pošlete složitý dotaz `SELECT`, databázový engine ho nezačne hned slepě vykonávat. Nejprve nastoupí komponenta zvaná **Optimalizátor dotazů (Query Optimizer)**.

- **Role optimalizátoru:** Optimalizátor analyzuje váš dotaz, podívá se na vnitřní statistiky tabulek (kolik má tabulka řádků, jak moc jsou hodnoty ve sloupcích unikátní) a vygeneruje několik variant, jak dotaz provést. Následně vybere tu nejlevnější variantu (s nejnižšími I/O operacemi a spotřebou CPU) a vytvoří **Prováděcí plán (Execution Plan)**.
- **Příkaz EXPLAIN (Klíč k optimalizaci v praxi):** Pokud vám nějaký dotaz běží pomalu, můžete před samotný příkaz `SELECT` napsat klíčové slovo `EXPLAIN`. Databáze dotaz nevykoná, ale vypíše vám tabulku s podrobným popisem zvoleného prováděcího plánu.

V něm se sledují kritické sloupce:

- `type` : Udává, jakým způsobem bude databáze tabulku prohledávat.
    - `ALL` : Nejhorší scénář (**Full Table Scan**) – databáze nevyužila žádný index.
    - `index` : Prochází se celý index (lepší než `ALL`, ale stále pomalé, pokud se musí číst celý index).
    - `range` : Vyhledávání v rozsahu pomocí indexu (dobré).
    - `ref` / `eq_ref` : Vyhledávání konkrétní hodnoty přes cizí/primární klíč (velmi rychlé).
    - `const` : Tabulka má maximálně jeden odpovídající řádek, hodnota je brána jako konstanta (ideální stav).
  - `possible_keys` : Seznam indexů, které by optimalizátor pro tento dotaz mohl teoreticky použít.
  - `key` : Název indexu, který optimalizátor reálně vybral a použije. Pokud je zde `NULL`, index se nepoužil.
  - `rows` : Odhadovaný počet řádků, které bude muset databáze prohledat, než najde výsledek.
- **Základní pravidla pro optimalizaci SQL dotazů:**

- **Indexujte cizí klíče:** Sloupce, přes které spojujete tabulky v `JOIN` podmínkách, by měly mít vždy index.
- **Vyhnete se `SELECT *`:** Vždy specifikujte jen ty sloupce, které reálně potřebujete. Snižuje to objem přenášených dat po síti a umožňuje databázi využít tzv. **Covering Index** (kdy jsou všechna požadovaná data přímo v indexu a databáze nemusí vůbec sahat do hlavní tabulky).
- **Pozor na "divoké" `LIKE`:** Podmínka `LIKE 'Prague%'` index využít dokáže (hledá se fixní začátek textu). Nicméně podmínka `LIKE '%Prague%'` (s procentem na začátku) index kompletně znefunkční a vynutí Full Table Scan.
- **Nepoužívejte funkce nad indexovanými sloupci ve `WHERE`:** Zápis `WHERE YEAR(datum_narozeni) = 2000` způsobí, že databáze musí funkci `YEAR()` spočítat pro každý řádek tabulky, takže index nepoužije. Správný optimalizovaný zápis je: `WHERE datum_narozeni BETWEEN '2000-01-01' AND '2000-12-31'`.

## Typické zkouškové otázky

### 1. Definujte index a vysvětlete jeho hlavní výhody a nevýhody.

- **Odpověď:** Index je pomocná datová struktura, která ukládá seřazené hodnoty z jednoho nebo více sloupců tabulky spolu s ukazateli na odpovídající řádky v datové tabulce.
- **Výhody:** Dramaticky zrychluje operace čtení (vyhledávání pomocí `WHERE`, řazení `ORDER BY`, spojování `JOIN`).
- **Nevýhody:** Zpomaluje operace zápisu (`INSERT`, `UPDATE`, `DELETE`), protože indexy musí být aktualizovány. Také zabírají dodatečné místo na disku a v paměti.

### 2. Popište rozdíl mezi B-stromovým a hašovacím indexem a uveďte, kdy je který vhodný.

- **Odpověď:**
  - **B-stromový index:** Je to vyvážená stromová struktura, kde uzly obsahují rozsahy hodnot a listové uzly klíče s ukazateli na data.

Listové uzly jsou často propojeny pro efektivní procházení. Má logaritmickou časovou složitost  $O(\log n)$ .

- **Hašovací index:** Využívá hašovací funkci k mapování hodnot sloupce na adresy (buckets). V ideálním případě má konstantní časovou složitost  $O(1)$ .

- **Vhodnost:**

- **B-stromový index** je univerzální a vhodný pro přesné shody ( = ), rozsahové dotazy ( < , > , BETWEEN ) a řazení ( ORDER BY ), protože zachovává uspořádání dat.
- **Hašovací index** je vhodný pouze pro velmi rychlé přesné shody ( = , IN ), ale je nepoužitelný pro rozsahové dotazy nebo řazení, jelikož hašovací funkce ničí informaci o uspořádání.

### 3. Vysvětlete pojmy clustered a non-clustered index. Kolik clustered indexů může mít tabulka?

- **Odpověď:**

- **Clustered index (primární index):** Fyzicky určuje pořadí, v jakém jsou řádky tabulky uloženy na disku. Data tabulky jsou fyzicky seřazena podle klíče tohoto indexu. Listy clustered indexu obsahují přímo data řádků.
- **Non-clustered index (sekundární index):** Je to samostatná datová struktura, která je uložena odděleně od dat tabulky. Obsahuje klíče a ukazatele na odpovídající řádky v tabulce (často odkazem na primární klíč, pokud existuje clustered index, nebo na fyzickou adresu řádku).

- **Počet clustered indexů:** Každá tabulka může mít maximálně jeden clustered index, protože řádky mohou být fyzicky seřazeny pouze jedním způsobem.

### 4. Jak databázový optimalizátor vybírá prováděcí plán a jak můžete tento plán analyzovat? Uveďte alespoň dvě pravidla pro optimalizaci SQL dotazů.

- **Odpověď:**

- **Výběr prováděcího plánu:** Databázový optimalizátor analyzuje SQL dotaz, využívá interní statistiky o datech v tabulkách (např.

počet řádků, distribuce hodnot, existence indexů) a generuje několik možných prováděcích plánů. Následně vybere ten plán, který odhaduje jako nejefektivnější z hlediska I/O operací a spotřeby CPU.

- **Analýza plánu:** Prováděcí plán lze analyzovat pomocí příkazu `EXPLAIN` (nebo `EXPLAIN ANALYZE`). Tento příkaz zobrazí detaily o tom, jak databáze zamýšlí dotaz vykonat, včetně typu přístupu k tabulkám (např. `ALL` pro Full Table Scan, `range` pro indexový rozsah), použitých indexů ( `key` ) a odhadovaného počtu prohledaných řádků ( `rows` ).
- **Pravidla pro optimalizaci:**
  1. **Indexujte cizí klíče:** Zrychluje `JOIN` operace.
  2. **Vyhněte se `SELECT *`:** Specifikujte jen potřebné sloupce, což může vést k využití Covering Indexu a snížení objemu dat.
  3. **Nepoužívejte funkce nad indexovanými sloupci ve `WHERE`:** Funkce znemožňují použití indexu (např. `WHERE YEAR(datum) = 2000` by mělo být `WHERE datum BETWEEN '2000-01-01' AND '2000-12-31'` ).

---

## Shrnutí kapitoly

Indexování a optimalizace dotazů jsou klíčové pro výkon relačních databází. Indexy, zejména B-stromové, dramaticky zrychlují čtení dat, ale za cenu zpomalení zápisu a zvýšené spotřeby úložiště. Databázový optimalizátor automaticky vybírá nejlepší prováděcí plán, který lze analyzovat pomocí příkazu `EXPLAIN`. Efektivní optimalizace dotazů vyžaduje znalost typů indexů (clustered, non-clustered, složené) a dodržování osvědčených postupů, jako je indexování cizích klíčů, selektivní výběr sloupců a vyhýbání se funkcím v `WHERE` klauzulích nad indexovanými sloupci.